



# Database Systems

## Design and Implementation



with Professor Joy Arulraj

# Importance of Database Systems



# Importance of Database Systems

Banking



# Importance of Database Systems

Banking

Healthcare



# Importance of Database Systems



Banking

Healthcare

Airlines

# Importance of Database Systems



Banking

Healthcare

Airlines

E-Commerce

# Why take this course?



# Why take this course?

Curiosity



# Why take this course?

Curiosity

Scalability



# Why take this course?

Curiosity

Scalability

Efficiency



# Why take this course?

Curiosity

Scalability

Efficiency

Versatility



# Why take this course?



# Why take this course?

## Storage Management



# Why take this course?

Storage Management

Query Optimization



# Why take this course?

Storage Management

Query Optimization

Index Structures



# Why take this course?

Storage Management

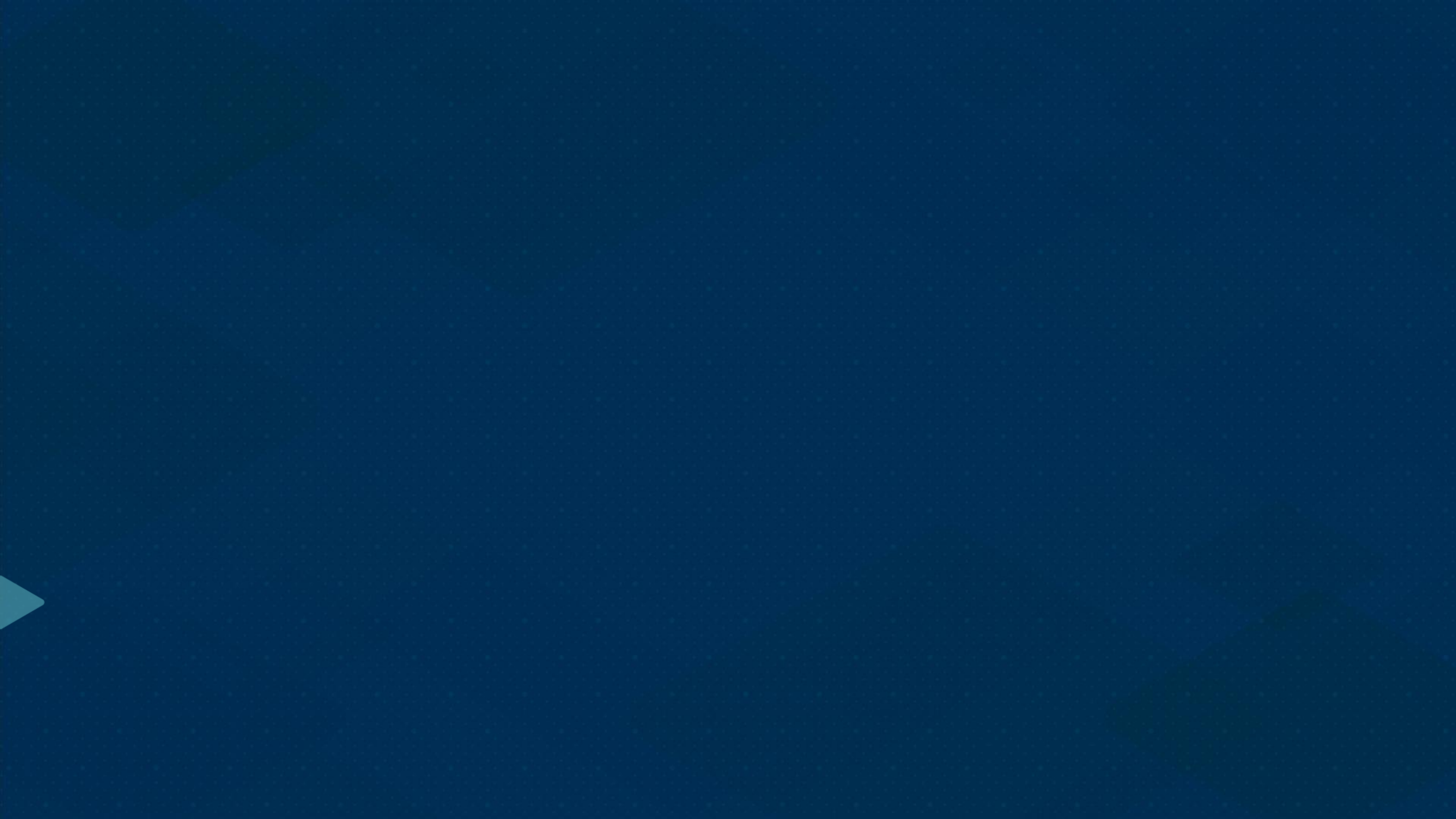
Query Optimization

Index Structures

SIMD Instructions



# Database System Introduction





**Illustrative  
Database  
Application**



**Illustrative  
Database  
Application**

The diagram consists of two yellow rounded rectangular boxes with gold borders, positioned side-by-side on a dark blue background. The left box contains the text 'Illustrative Database Application' and the right box contains the text 'Flat-File Database System'. A light blue arrow points from the left edge of the slide towards the boxes.

**Flat-File  
Database  
System**

**Illustrative  
Database  
Application**

**Flat-File  
Database  
System**

**Flat-File  
Database  
Limitations**

# Social Media Analytics Application

Social  
Media  
Analytics

# Social Media Analytics Application

Social  
Media  
Analytics

Social Trends

# Social Media Analytics Application

Social  
Media  
Analytics

Social Trends

Sentiments

# Social Media Analytics Application

Social  
Media  
Analytics

Social Trends

Sentiments

Interactions

# Social Media Analytics Application

Data

# Social Media Analytics Application

Data

Users.txt

# Social Media Analytics Application

Data

Users.txt

Posts.txt

# Social Media Analytics Application

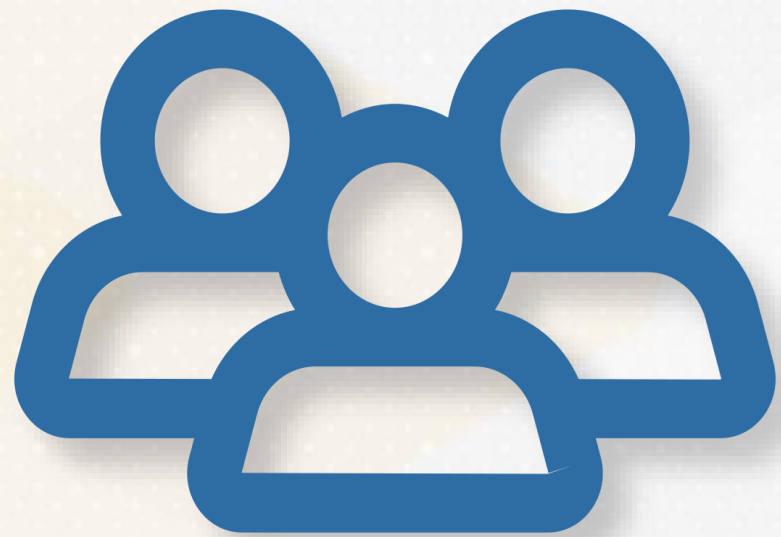
Data

Users.txt

Posts.txt

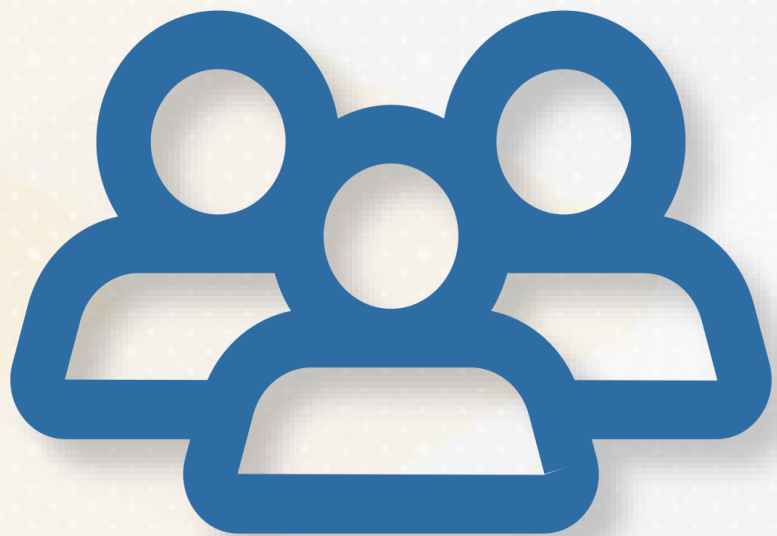
Interactions.txt

# Users Text File



**Users.txt**

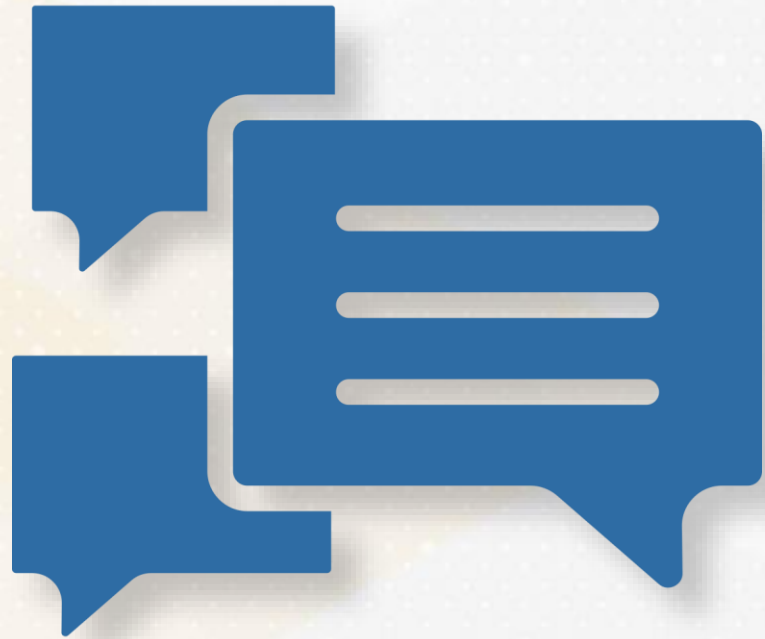
# Users Text File



**Users.txt**

| UserName,          | Location    |
|--------------------|-------------|
| Timothée Chalamet, | Paris       |
| Lana Condor,       | Los Angeles |
| Liu Yifei,         | Beijing     |
| Burna Boy,         | Lagos       |
| Kriti Sanon,       | Mumbai      |

# Posts Text File



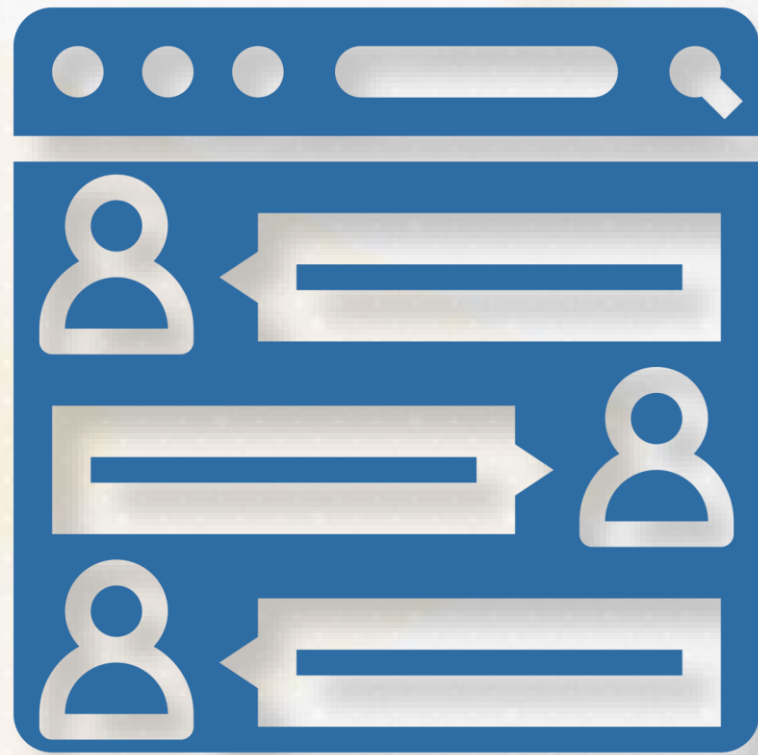
**Posts.txt**

# Posts Text File



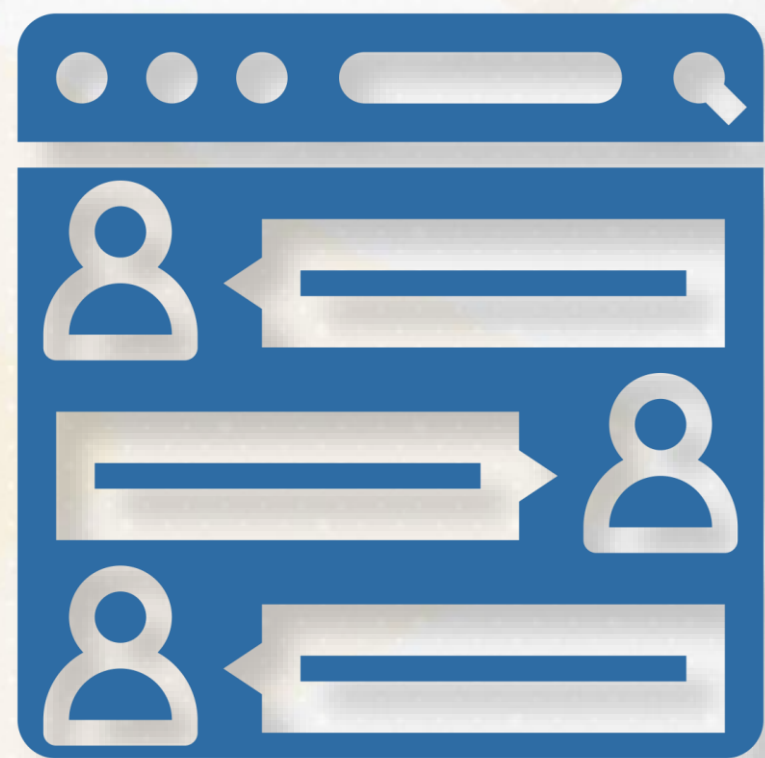
| PostID, | UserName,          | PostContent                                                                                                                                                                                                     |
|---------|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1001,   | Timothée Chalamet, | Excited to start filming my new movie!                                                                                                                                                                          |
| 1002,   | Lana Condor,       | Had a great time at the beach today!<br>  |
| 1003,   | Liu Yifei,         | Enjoying the scenery in Beijing!                                                                                           |
| 1004,   | Burna Boy,         | Live performance tonight in Lagos!    |
| 1005,   | Kriti Sanon,       | Loving the vibrant energy of Mumbai!                                                                                       |

# Interactions Text File



**Interactions.txt**

# Interactions Text File

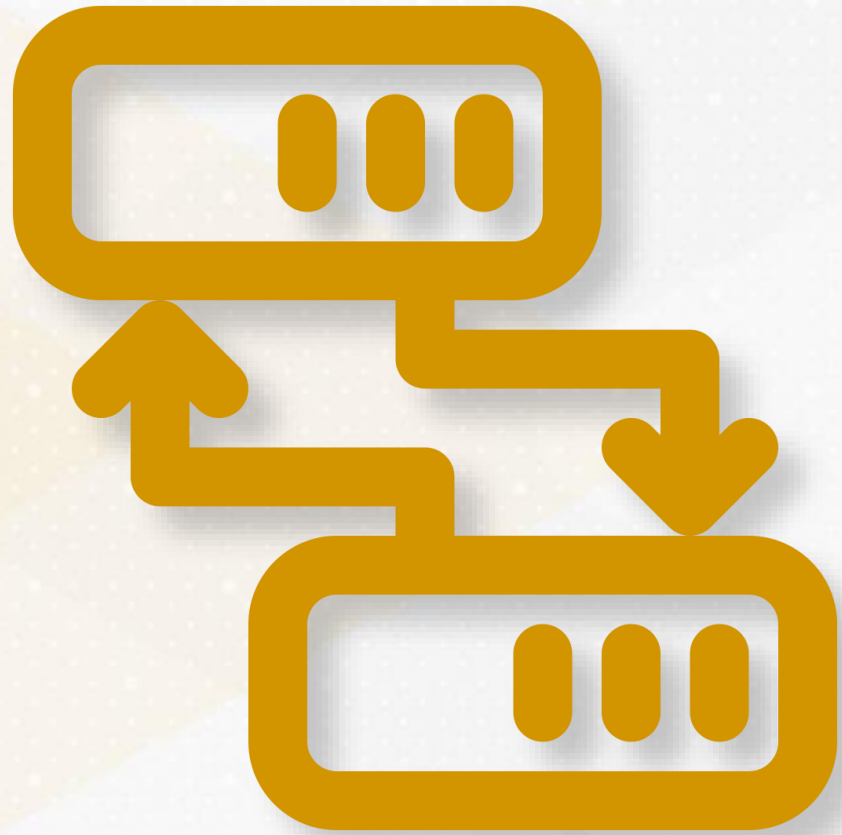


Interactions.txt

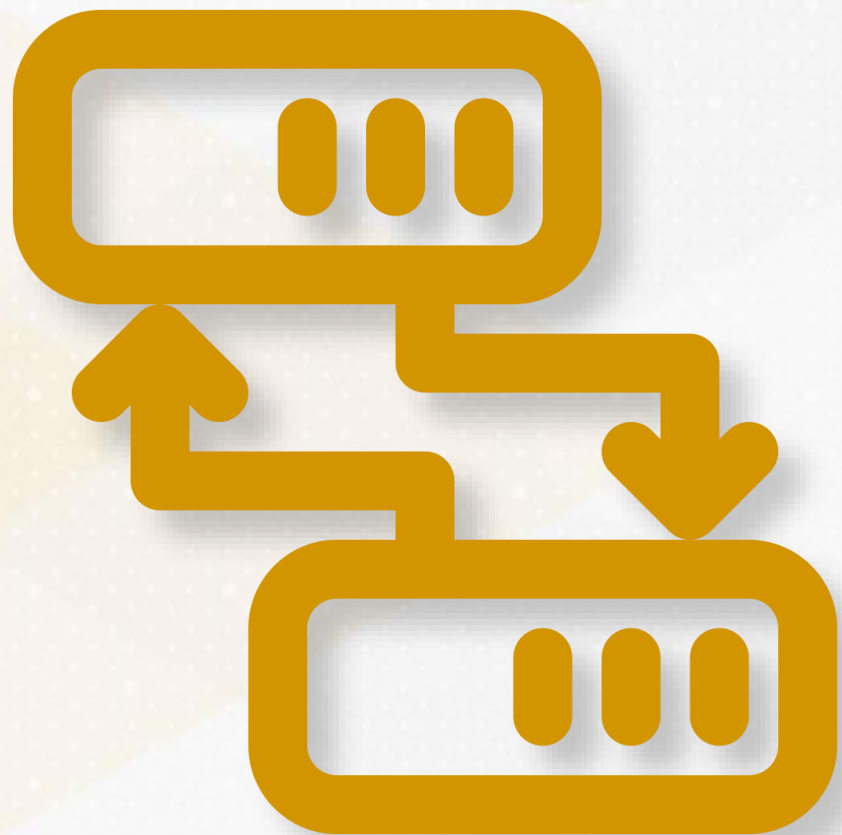
| PostID, | UserName,    | Reaction Type, | Comment                |
|---------|--------------|----------------|------------------------|
| 1001,   | Lana Condor, | Comment,       | Love it!               |
| 1002,   | Liu Yifei,   | Like,          | -                      |
| 1003,   | Burna Boy    | Like,          | -                      |
| 1004,   | Kriti Sanon  | Comment,       | Wish I could be there! |

# Flat-File Database Limitations

# Limitation #1: Data Redundancy



# Limitation #1: Data Redundancy



| PostID, | UserName,          | PostContent                            |
|---------|--------------------|----------------------------------------|
| 1001,   | Timothée Chalamet, | Excited to start filming my new movie! |
| 1006,   | Timothée Chalamet, | Exploring the streets of Paris! FR     |
| 1007,   | Timothée Lamet,    | Just wrapped up a day of filming 🎬     |
| 1008,   | Timothée Chalamet, | Any book recommendations?              |

# Limitation #2: Slow Operations

# Limitation #2: Slow Operations

| <b>UserName</b>    | <b>Location</b> |
|--------------------|-----------------|
| Timothée Chalamet, | Paris           |
| Lana Condor,       | Los Angeles     |
| Liu Yifei,         | Beijing         |
| Burna Boy,         | Lagos           |
| Kriti Sanon,       | Mumbai          |

## Limitation #2: Slow Operations

| UserName                | Location               |
|-------------------------|------------------------|
| Timothée Chalamet,      | Paris                  |
| <del>Lana Condor,</del> | <del>Los Angeles</del> |
| Liu Yifei,              | Beijing                |
| Burna Boy,              | Lagos                  |
| Kriti Sanon,            | Mumbai                 |



# Limitation #2: Slow Operations

| UserName                | Location               |
|-------------------------|------------------------|
| Timothée Chalamet,      | Paris                  |
| <del>Lana Condor,</del> | <del>Los Angeles</del> |
| Liu Yifei,              | Beijing                |
| Burna Boy,              | Lagos                  |
| Kriti Sanon,            | Mumbai                 |



# Limitation #2: Slow Operations

| <b>UserName</b>    | <b>Location</b> |
|--------------------|-----------------|
| Timothée Chalamet, | Paris           |
| Lana Condor,       | Los Angeles     |
| Liu Yifei,         | Beijing         |
| Burna Boy,         | Lagos           |
| Kriti Sanon,       | Mumbai          |

# Limitation #3: Slow Queries

| UserName,          | Location    |
|--------------------|-------------|
| Timothée Chalamet, | Paris       |
| Lana Condor,       | Los Angeles |
| Liu Yifei,         | Beijing     |
| Burna Boy,         | Lagos       |
| Kriti Sanon,       | Mumbai      |

# Limitation #3: Slow Queries

| UserName,          | Location    |
|--------------------|-------------|
| Timothée Chalamet, | Paris       |
| Lana Condor,       | Los Angeles |
| Liu Yifei,         | Beijing     |
| Burna Boy,         | Lagos       |
| Kriti Sanon,       | Mumbai      |

# Limitation #4: Concurrent Updates

# Limitation #4: Concurrent Updates

USER 1



Timothée Chalamet,  
Paris

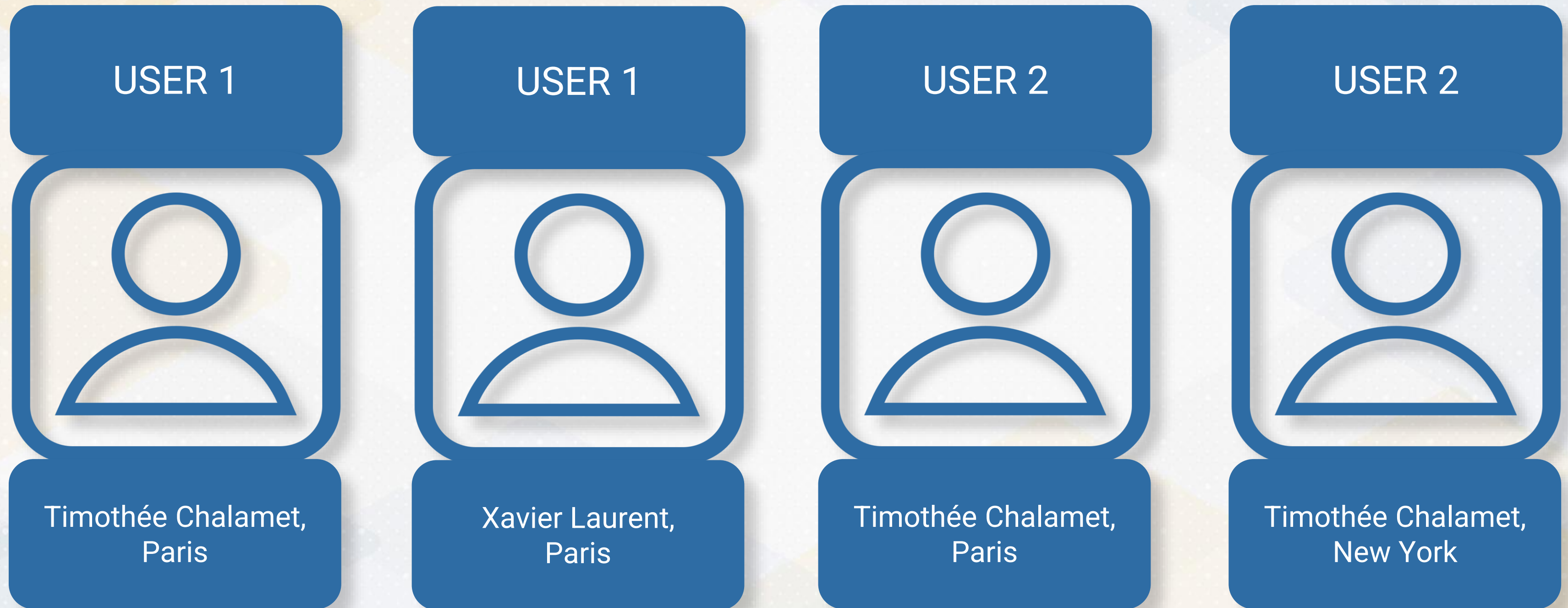
# Limitation #4: Concurrent Updates



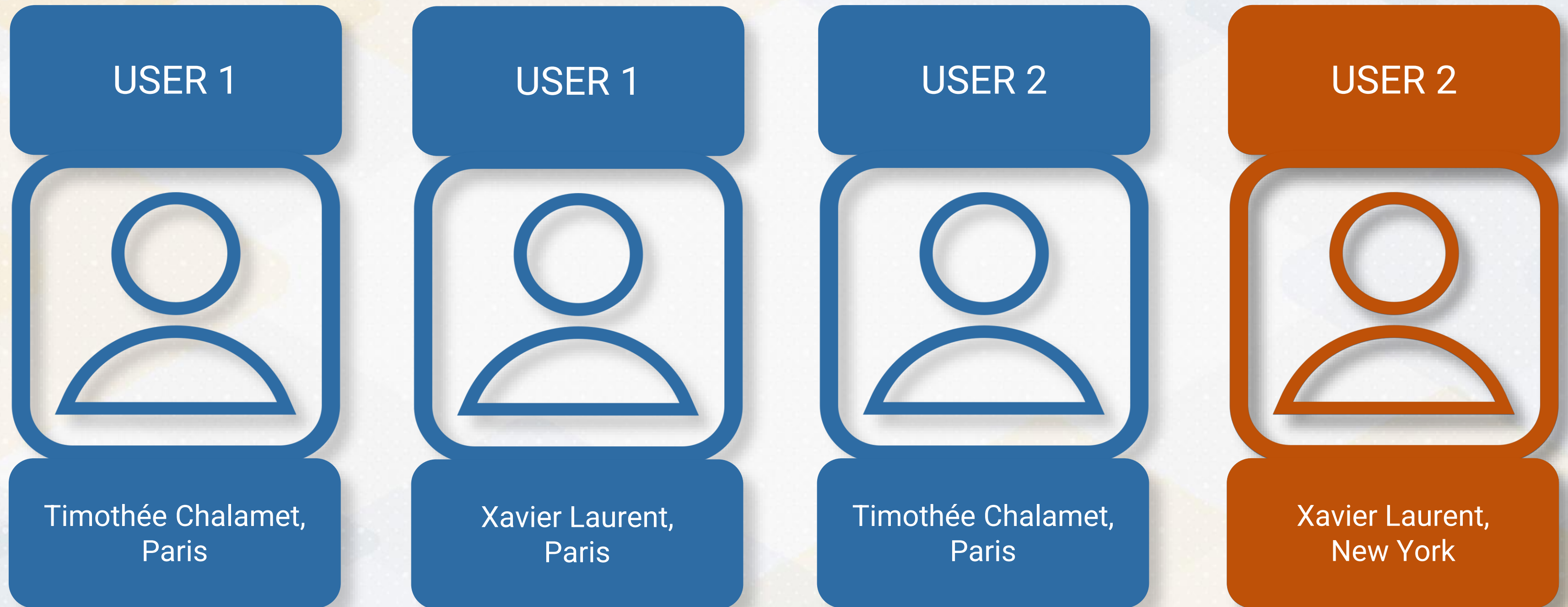
# Limitation #4: Concurrent Updates



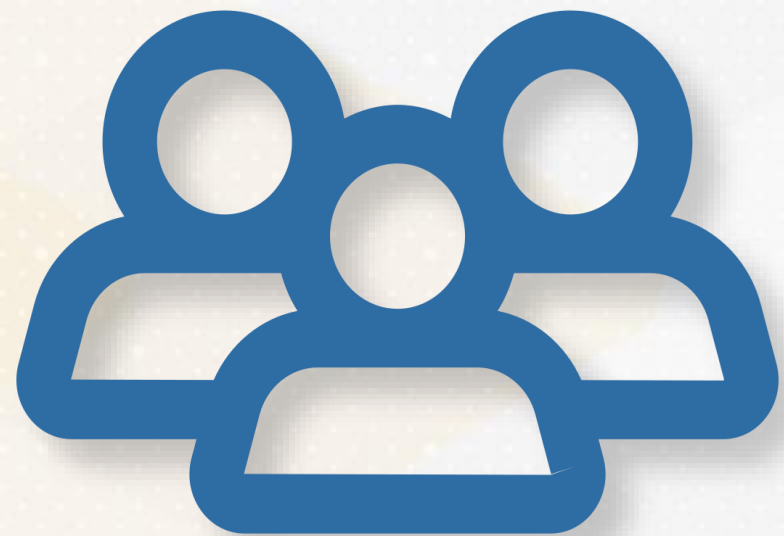
# Limitation #4: Concurrent Updates



# Limitation #4: Concurrent Updates



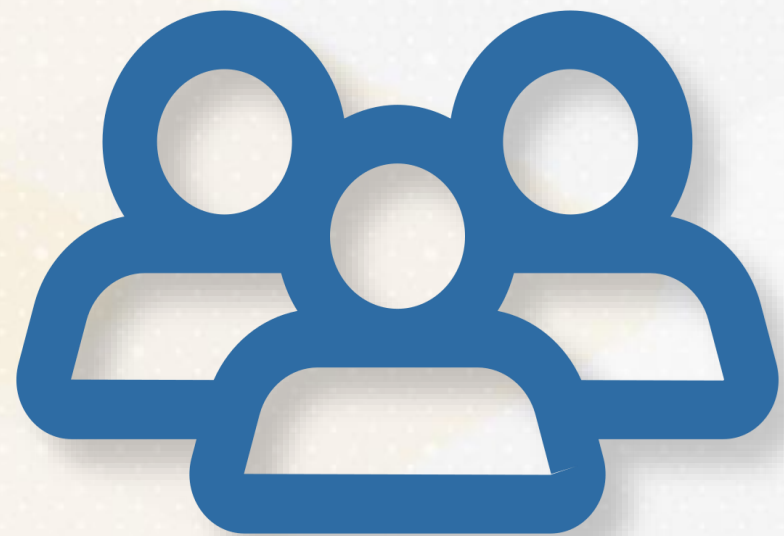
# Limitation #5: Handling Disk Failure



**Users.txt**

| UserName,          | Location,    | Country |
|--------------------|--------------|---------|
| Timothée Chalamet, | Paris,       |         |
| Lana Condor,       | Los Angeles, |         |
| Liu Yifei,         | Beijing,     |         |
| Burna Boy,         | Lagos        |         |
| Kriti Sanon,       | Mumbai       |         |

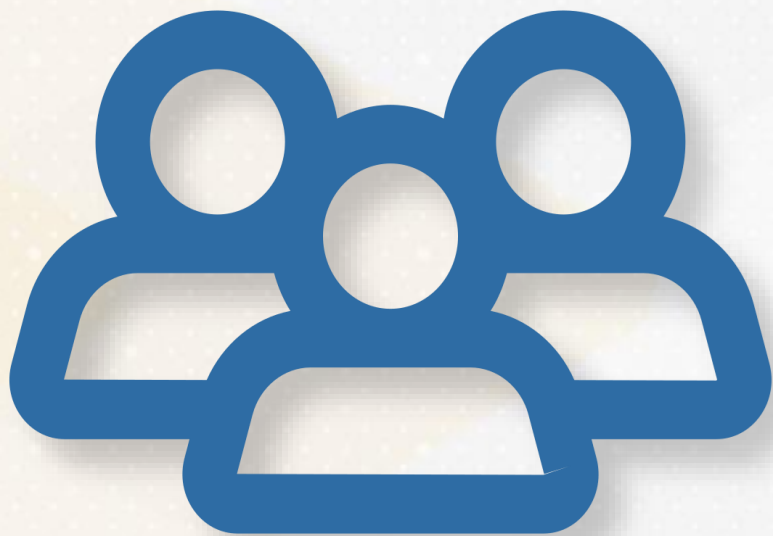
# Limitation #5: Handling Disk Failure



**Users.txt**

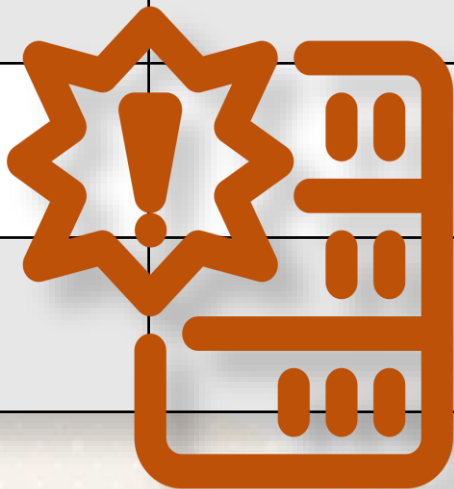
| UserName,          | Location,    | Country |
|--------------------|--------------|---------|
| Timothée Chalamet, | Paris,       | France  |
| Lana Condor,       | Los Angeles, | USA     |
| Liu Yifei,         | Beijing,     | China   |
| Burna Boy,         | Lagos        |         |
| Kriti Sanon,       | Mumbai       |         |

# Limitation #5: Handling Disk Failure



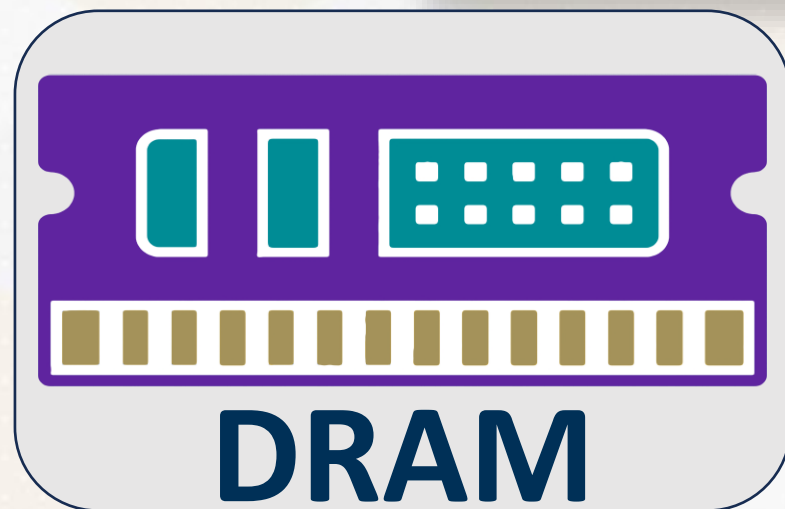
Users.txt

| UserName,          | Location,    | Country |
|--------------------|--------------|---------|
| Timothée Chalamet, | Paris,       | France  |
| Lana Condor,       | Los Angeles, | USA     |
| Liu Yifei,         | Beijing,     | China   |
| Burna Boy,         | Lagos        |         |
| Kriti Sanon,       | Mumbai       |         |



# Limitation #6: Memory Management

*Faster access - not durable*



|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |

*Cached Pages*



|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

*Database*

*Slower access - but durable*

# Limitation #7: Usability

# Limitation #7: Usability

Custom Code

# Limitation #7: Usability

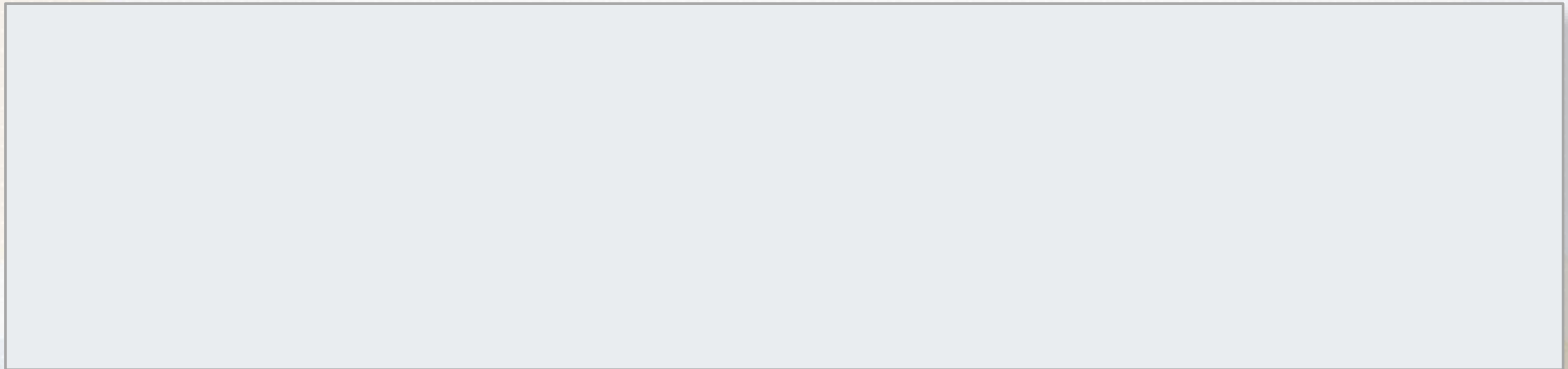
Custom Code

Comments Query Code

# Limitation #7: Usability

Custom Code

Comments Query Code



# Limitation #7: Usability

Custom Code

Comments Query Code

```
def get_comments_by_user(file_path, user_name):
```

# Limitation #7: Usability

Custom Code

Comments Query Code

```
def get_comments_by_user(file_path, user_name):  
    comments = []
```

# Limitation #7: Usability

Custom Code

Comments Query Code

```
def get_comments_by_user(file_path, user_name):  
    comments = []  
    with open(file_path, 'r') as file:
```

# Limitation #7: Usability

Custom Code

Comments Query Code

```
def get_comments_by_user(file_path, user_name):  
    comments = []  
    with open(file_path, 'r') as file:  
        for line in file:
```

# Limitation #7: Usability

Custom Code

Comments Query Code

```
def get_comments_by_user(file_path, user_name):  
    comments = []  
    with open(file_path, 'r') as file:  
        for line in file:  
            post_id, user, reaction_type, comment_text = line.strip().split(',')
```

# Limitation #7: Usability

Custom Code

Comments Query Code

```
def get_comments_by_user(file_path, user_name):  
    comments = []  
    with open(file_path, 'r') as file:  
        for line in file:  
            post_id, user, reaction_type, comment_text = line.strip().split(',')  
            if user == user_name and reaction_type == "Comment":
```

# Limitation #7: Usability

Custom Code

Comments Query Code

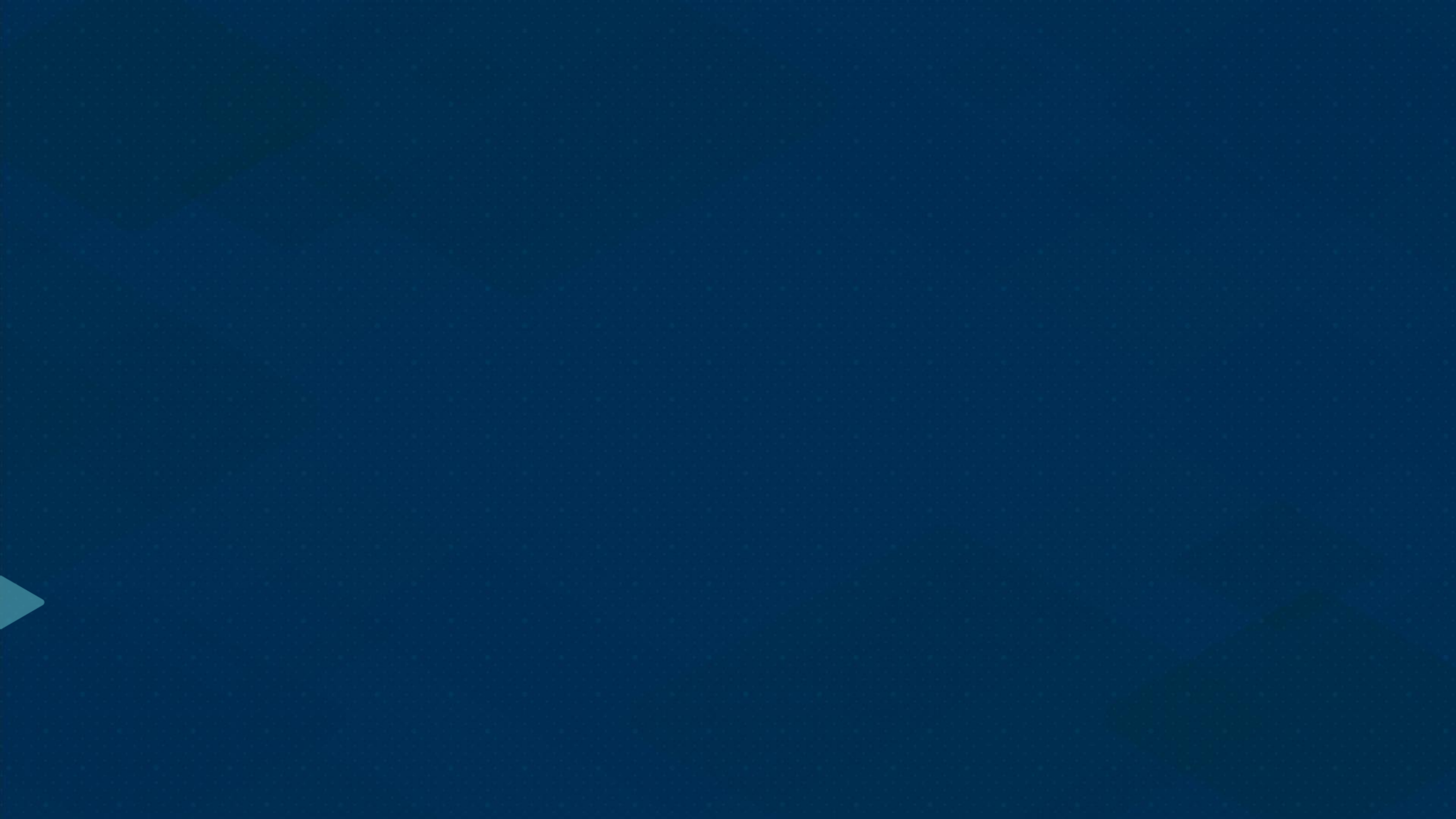
```
def get_comments_by_user(file_path, user_name):  
    comments = []  
    with open(file_path, 'r') as file:  
        for line in file:  
            post_id, user, reaction_type, comment_text = line.strip().split(',')  
            if user == user_name and reaction_type == "Comment":  
                comments.append((post_id, comment_text))
```

# Limitation #7: Usability

Custom Code

Comments Query Code

```
def get_comments_by_user(file_path, user_name):  
    comments = []  
    with open(file_path, 'r') as file:  
        for line in file:  
            post_id, user, reaction_type, comment_text = line.strip().split(',')  
            if user == user_name and reaction_type == "Comment":  
                comments.append((post_id, comment_text))  
    return comments
```



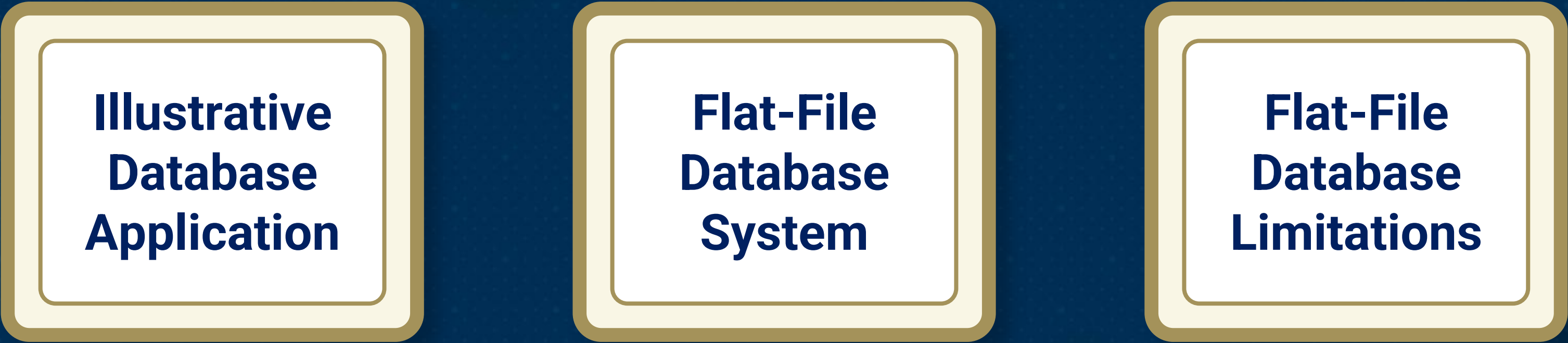


**Illustrative  
Database  
Application**

**Illustrative  
Database  
Application**

The diagram consists of two yellow rounded rectangular boxes with gold borders, positioned side-by-side on a dark blue background. The left box contains the text 'Illustrative Database Application' and the right box contains the text 'Flat-File Database System'. A light blue arrow points from the left box towards the right box, indicating a flow or relationship between the two.

**Flat-File  
Database  
System**

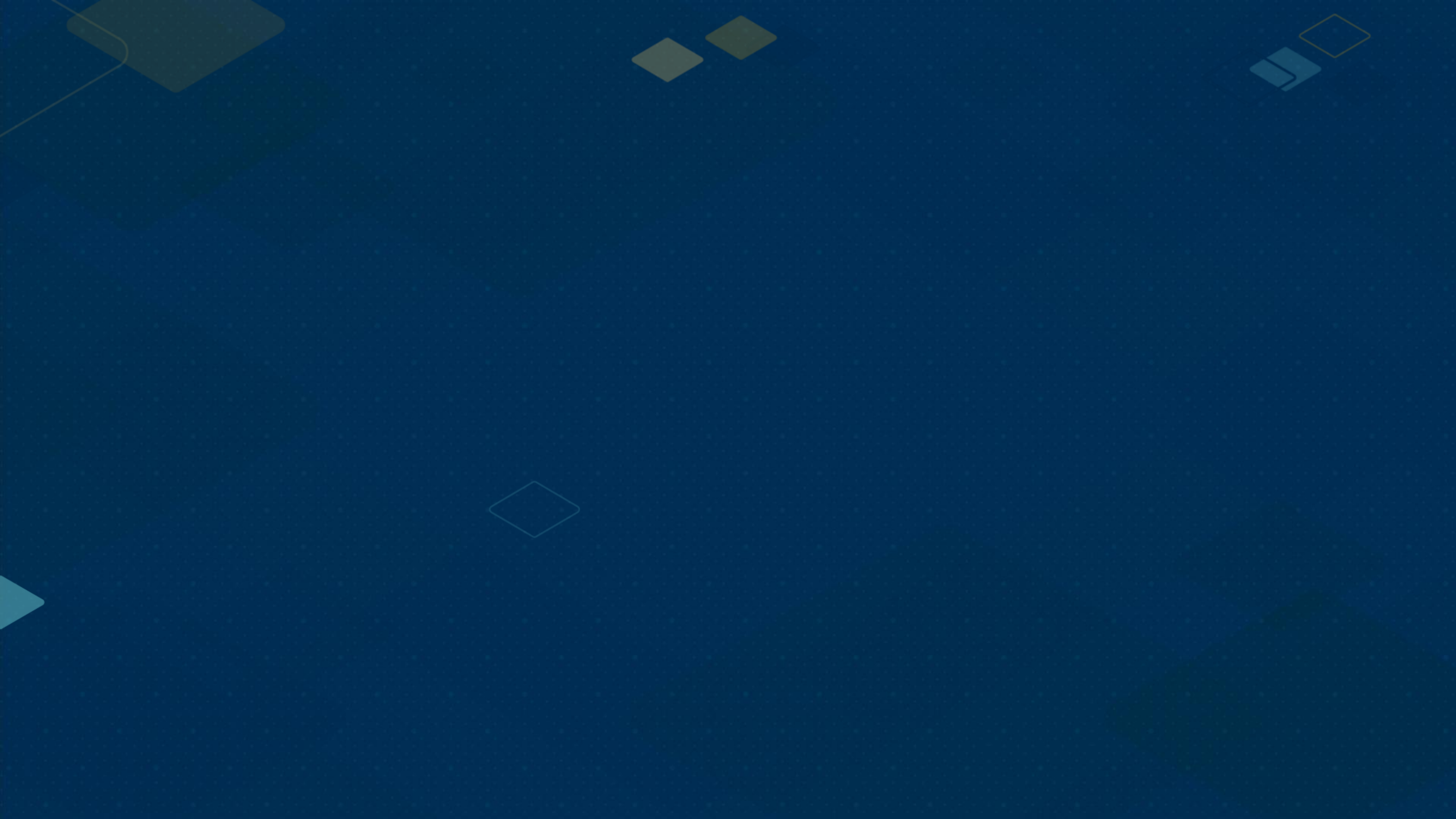


**Illustrative  
Database  
Application**

**Flat-File  
Database  
System**

**Flat-File  
Database  
Limitations**

# Relational Database





# **Relational Database**

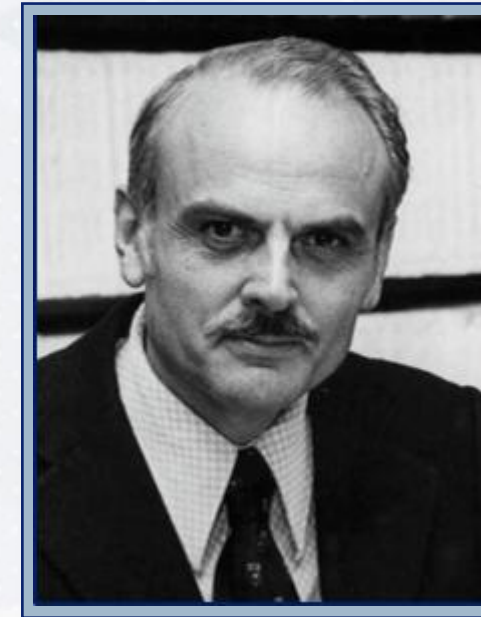


**Relational  
Database**

**Relational  
Database  
Benefits**

# Relational Database

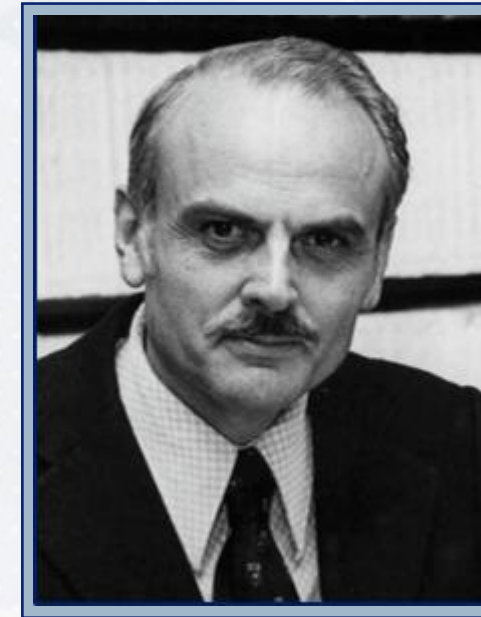
**Ted Codd (1970)**



|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Relational Database

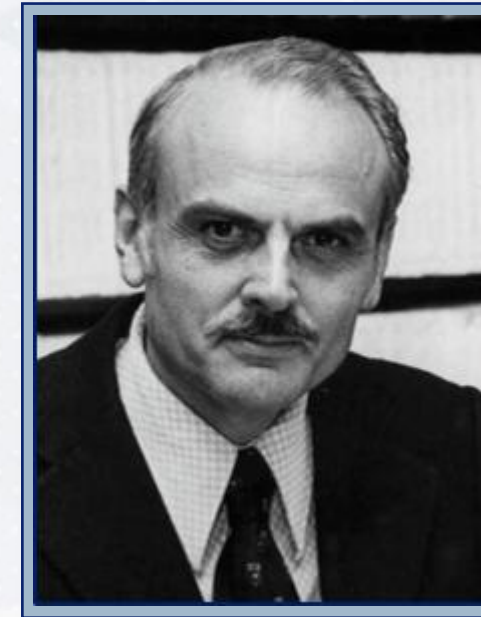
**Ted Codd (1970)**



❖ Scientist at IBM

# Relational Database

**Ted Codd (1970)**

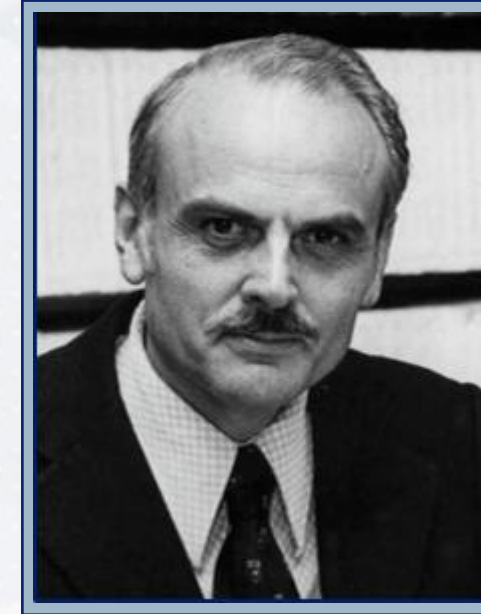


❖ Scientist at IBM

❖ Simplify Data Management

# Relational Database

**Ted Codd (1970)**



- ❖ Scientist at IBM
- ❖ Simplify Data Management
- ❖ Organize Data as Tables

# Relational Database

| UserName          | Location    |
|-------------------|-------------|
| Timothée Chalamet | Paris       |
| Lana Condor       | Los Angeles |
| Liu Yifei         | Beijing     |
| Burna Boy         | Lagos       |
| Kriti Sanon       | Mumbai      |

# Relational Database

**Row/  
Tuple**



| UserName          | Location    |
|-------------------|-------------|
| Timothée Chalamet | Paris       |
| Lana Condor       | Los Angeles |
| Liu Yifei         | Beijing     |
| Burna Boy         | Lagos       |
| Kriti Sanon       | Mumbai      |

# Relational Database

**Column / Attribute**



**Row/  
Tuple**



| UserName          | Location    |
|-------------------|-------------|
| Timothée Chalamet | Paris       |
| Lana Condor       | Los Angeles |
| Liu Yifei         | Beijing     |
| Burna Boy         | Lagos       |
| Kriti Sanon       | Mumbai      |

# Relational Database

**Column / Attribute**



**Row/  
Tuple**



| UserName          | Location    |
|-------------------|-------------|
| Timothée Chalamet | Paris       |
| Lana Condor       | Los Angeles |
| Liu Yifei         | Beijing     |
| Burna Boy         | Lagos       |
| Kriti Sanon       | Mumbai      |

# Relational Database

| UserID | UserName          | Location    |
|--------|-------------------|-------------|
| 1      | Timothée Chalamet | Paris       |
| 2      | Lana Condor       | Los Angeles |
| 3      | Liu Yifei         | Beijing     |
| 4      | Burna Boy         | Lagos       |
| 5      | Kriti Sanon       | Mumbai      |

| PostID | UserID | PostContent                              |
|--------|--------|------------------------------------------|
| 1001   | 1      | Excited to start filming my new movie!   |
| 1002   | 2      | Had a great time at the beach today! 🌊☀️ |
| 1003   | 3      | Enjoying the scenery in Beijing! 🏞️      |
| 1004   | 4      | Live performance tonight in Lagos! 🎤🎶    |
| 1005   | 5      | Loving the vibrant energy of Mumbai! 🏙️  |

| PostID | UserID | ReactionType | Content                |
|--------|--------|--------------|------------------------|
| 1001   | 2      | Comment      | Love it!               |
| 1002   | 3      | Like         | -                      |
| 1003   | 4      | Like         | -                      |
| 1004   | 5      | Comment      | Wish I could be there! |

# Relational Database

| UserID | UserName          | Location    |
|--------|-------------------|-------------|
| 1      | Timothée Chalamet | Paris       |
| 2      | Lana Condor       | Los Angeles |
| 3      | Liu Yifei         | Beijing     |
| 4      | Burna Boy         | Lagos       |
| 5      | Kriti Sanon       | Mumbai      |

Users

| PostID | UserID | PostContent                              |
|--------|--------|------------------------------------------|
| 1001   | 1      | Excited to start filming my new movie!   |
| 1002   | 2      | Had a great time at the beach today! 🌊☀️ |
| 1003   | 3      | Enjoying the scenery in Beijing! 🏞️      |
| 1004   | 4      | Live performance tonight in Lagos! 🎤🎶    |
| 1005   | 5      | Loving the vibrant energy of Mumbai! 🏙️  |

Posts

| PostID | UserID | ReactionType | Content                |
|--------|--------|--------------|------------------------|
| 1001   | 2      | Comment      | Love it!               |
| 1002   | 3      | Like         | -                      |
| 1003   | 4      | Like         | -                      |
| 1004   | 5      | Comment      | Wish I could be there! |

Interactions

# Relational Database

| Students | Grades |
|----------|--------|
|          |        |
|          |        |
|          |        |

# Relational Database

Mathematic  
Set Theory

| Students | Grades |
|----------|--------|
|          |        |
|          |        |
|          |        |

# Relational Database

Mathematic  
Set Theory

Data Sets  
Relationship

| Students | Grades |
|----------|--------|
|          |        |
|          |        |
|          |        |

# Relational Database

Mathematic  
Set Theory

Data Sets  
Relationship

Efficient Data  
Set Links

| Students | Grades |
|----------|--------|
|          |        |
|          |        |
|          |        |

# Relational Database

Mathematic  
Set Theory

Data Sets  
Relationship

Efficient Data  
Set Links

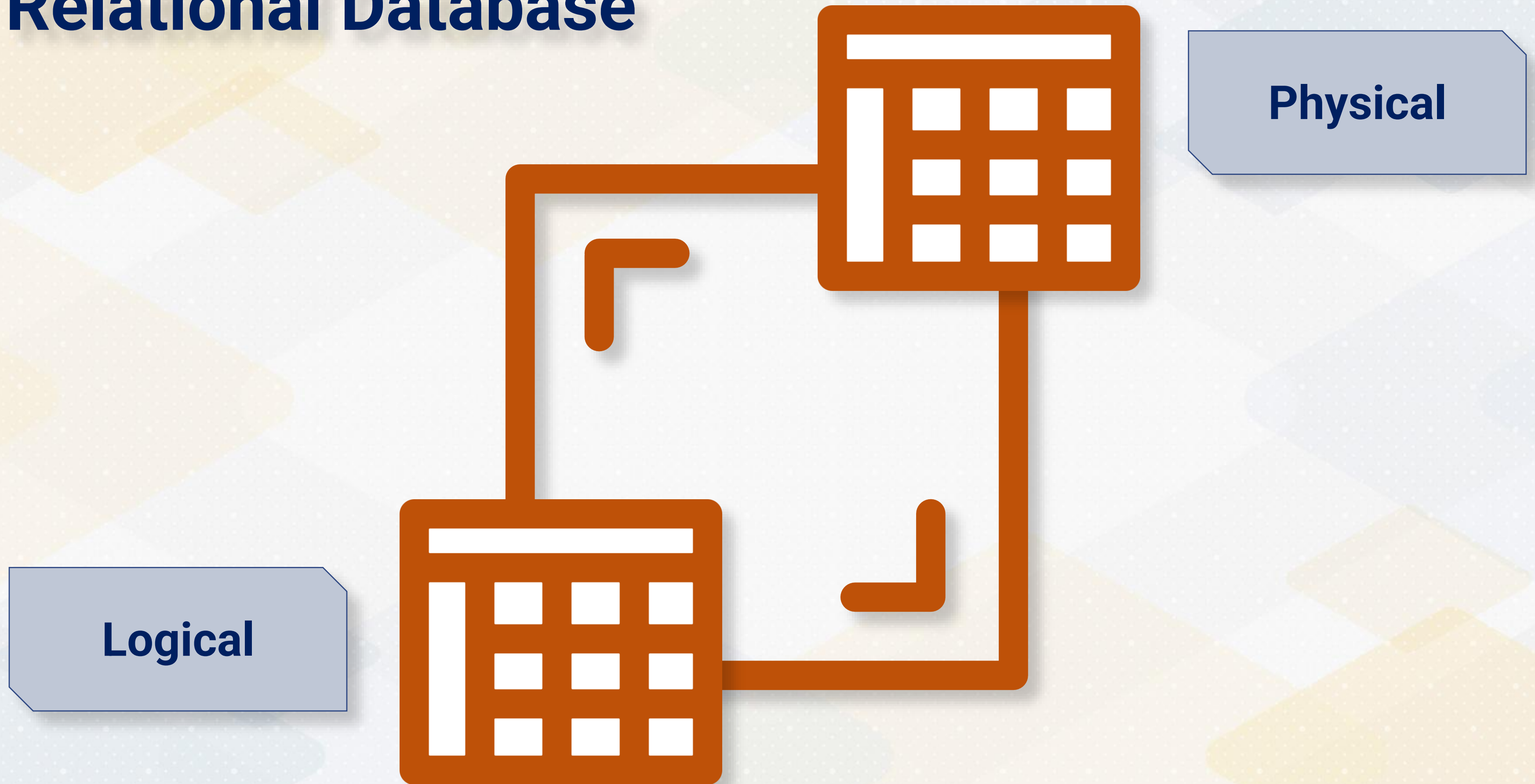
| Students | Grades |
|----------|--------|
| Alice    | B      |
| Bob      | A      |
| Charlie  | C      |

$R=\{(Alice,B),(Bob,A),(Charlie,C)\}$

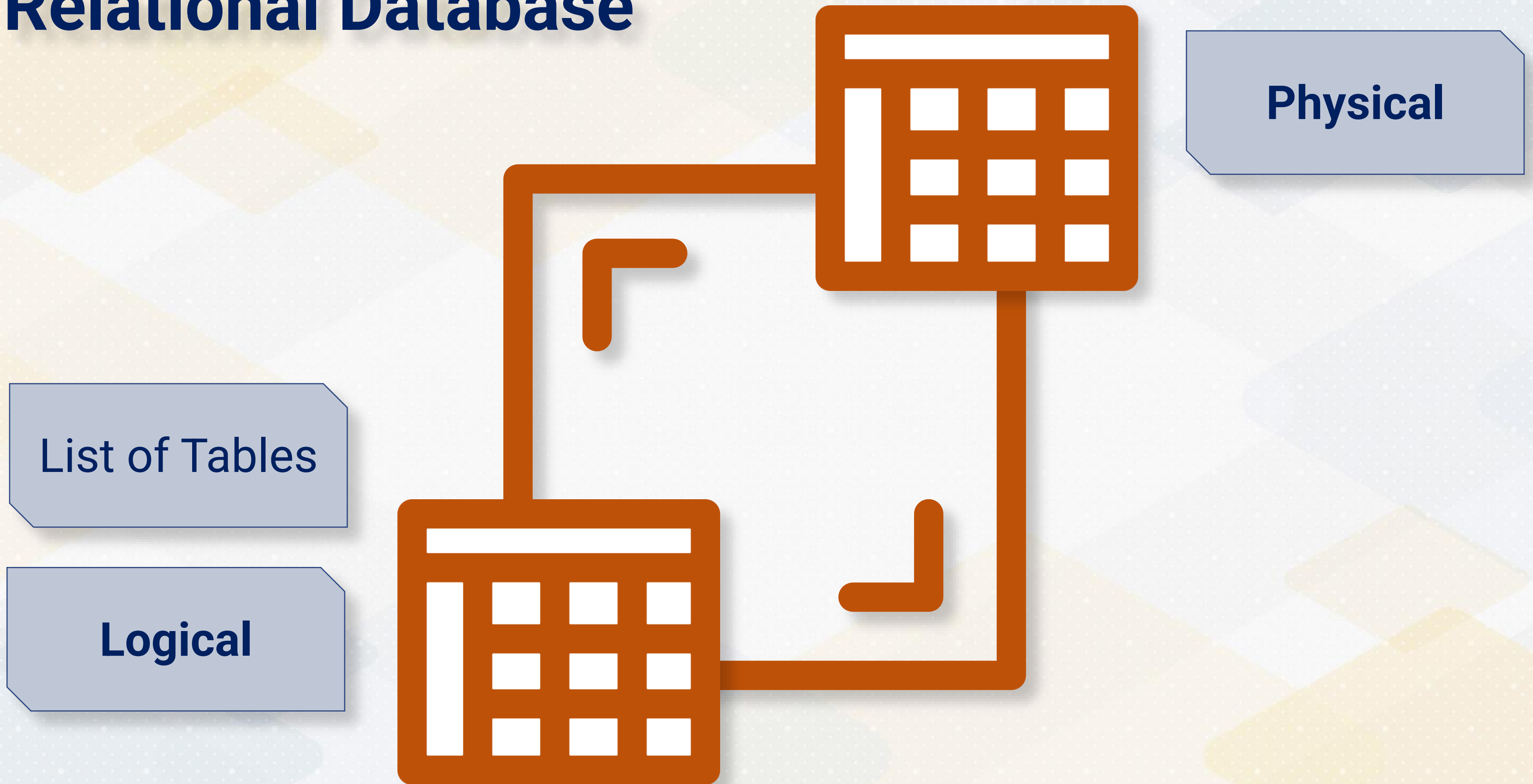
# Relational Database



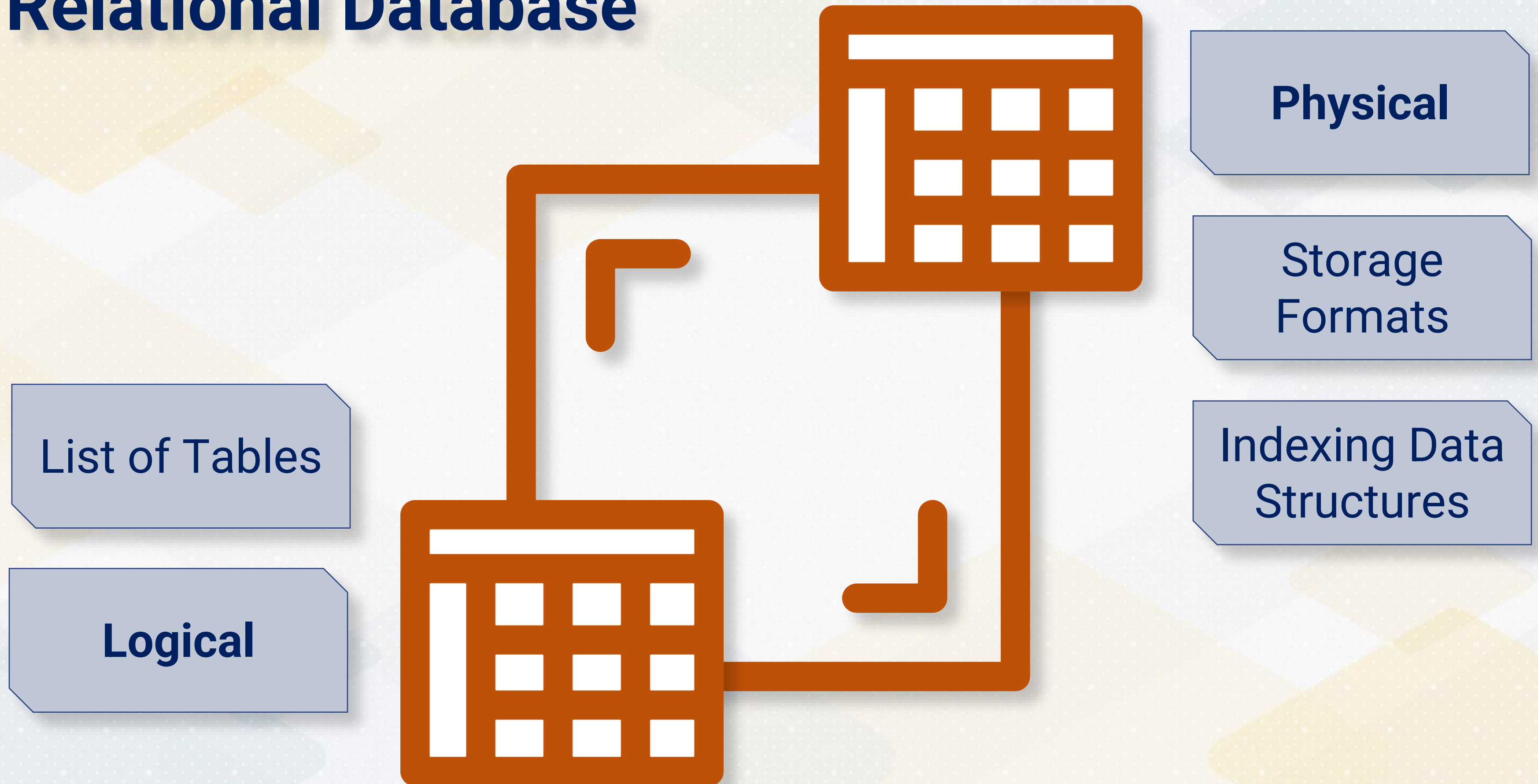
# Relational Database



# Relational Database



# Relational Database



# Relational Database

---

# Relational Database

Logical Database  
Design

---

Physical  
Database Design

# Relational Database

Logical Database  
Design

Simple Query Language for  
Complex Data Manipulation

---

Physical  
Database Design

# Relational Database

Logical Database  
Design

Simple Query Language for  
Complex Data Manipulation

---

Physical  
Database Design

Optimize Indexing for  
Storage Hardware

# Logical Database Design: Primary Key

| UserID | UserName          | Location    |
|--------|-------------------|-------------|
| 1      | Timothée Chalamet | Paris       |
| 2      | Lana Condor       | Los Angeles |
| 3      | Liu Yifei         | Beijing     |
| 4      | Burna Boy         | Lagos       |
| 5      | Kriti Sanon       | Mumbai      |

# Logical Database Design: Foreign Key

| UserID | UserName          | Location    |
|--------|-------------------|-------------|
| 1      | Timothée Chalamet | Paris       |
| 2      | Lana Condor       | Los Angeles |
| 3      | Liu Yifei         | Beijing     |
| 4      | Burna Boy         | Lagos       |
| 5      | Kriti Sanon       | Mumbai      |

| PostID | UserID | PostContent                              |
|--------|--------|------------------------------------------|
| 1001   | 1      | Excited to start filming my new movie!   |
| 1002   | 2      | Had a great time at the beach today! 🌊☀️ |
| 1003   | 3      | Enjoying the scenery in Beijing! 🏞️      |
| 1004   | 4      | Live performance tonight in Lagos! 🎤🎵    |
| 1005   | 5      | Loving the vibrant energy of Mumbai! 🏙️  |

# Logical Database Design: Foreign Key

| UserID | UserName          | Location    |
|--------|-------------------|-------------|
| 1      | Timothée Chalamet | Paris       |
| 2      | Lana Condor       | Los Angeles |
| 3      | Liu Yifei         | Beijing     |
| 4      | Burna Boy         | Lagos       |
| 5      | Kriti Sanon       | Mumbai      |

| PostID | UserID | PostContent                              |
|--------|--------|------------------------------------------|
| 1001   | 1      | Excited to start filming my new movie!   |
| 1002   | 2      | Had a great time at the beach today! 🌊☀️ |
| 1003   | 3      | Enjoying the scenery in Beijing! 🏞️      |
| 1004   | 4      | Live performance tonight in Lagos! 🎤🎵    |
| 1005   | 5      | Loving the vibrant energy of Mumbai! 🏙️  |

# Logical Database Design: Foreign Key

| UserID | UserName          | Location    |
|--------|-------------------|-------------|
| 1      | Timothée Chalamet | Paris       |
| 2      | Lana Condor       | Los Angeles |
| 3      | Liu Yifei         | Beijing     |
| 4      | Burna Boy         | Lagos       |
| 5      | Kriti Sanon       | Mumbai      |

| PostID | UserID | PostContent                               |
|--------|--------|-------------------------------------------|
| 1001   | 1      | Excited to start filming my new movie!    |
| 1002   | 2      | Had a great time at the beach today! 🌊 ☀️ |
| 1003   | 3      | Enjoying the scenery in Beijing! 🏞️       |
| 1004   | 4      | Live performance tonight in Lagos! 🎤 🎶    |
| 1005   | 5      | Loving the vibrant energy of Mumbai! 🏙️   |

# Logical Database Design: Foreign Key

| UserID | UserName          | Location    |
|--------|-------------------|-------------|
| 1      | Timothée Chalamet | Paris       |
| 2      | Lana Condor       | Los Angeles |
| 3      | Liu Yifei         | Beijing     |
| 4      | Burna Boy         | Lagos       |
| 5      | Kriti Sanon       | Mumbai      |

Referential  
Data  
Integrity

| PostID | UserID | PostContent                               |
|--------|--------|-------------------------------------------|
| 1001   | 1      | Excited to start filming my new movie!    |
| 1002   | 2      | Had a great time at the beach today! 🌊 ☀️ |
| 1003   | 3      | Enjoying the scenery in Beijing! 🏞️       |
| 1004   | 4      | Live performance tonight in Lagos! 🎤 🎵    |
| 1005   | 5      | Loving the vibrant energy of Mumbai! 🏙️   |



# Relational Database Benefits



# Benefit #1: No Data Redundancy

| UserID | UserName          | Location |
|--------|-------------------|----------|
| 1      | Timothée Chalamet | Paris    |

| PostID | UserID | PostContent                            |
|--------|--------|----------------------------------------|
| 1001   | 1      | Excited to start filming my new movie! |
| 1006   | 1      | Exploring the streets of Paris!        |
| 1007   | 1      | Just wrapped up a day of filming       |
| 1008   | 1      | Any book recommendations?              |

# Benefit #1: No Data Redundancy

| UserID | UserName              | Location |
|--------|-----------------------|----------|
| 1      | Sir Timothée Chalamet | Paris    |

| PostID | UserID | PostContent                            |
|--------|--------|----------------------------------------|
| 1001   | 1      | Excited to start filming my new movie! |
| 1006   | 1      | Exploring the streets of Paris!        |
| 1007   | 1      | Just wrapped up a day of filming       |
| 1008   | 1      | Any book recommendations?              |

# Benefit #2: Fast Operations



# Benefit #2: Fast Operations

❖ Efficient Data Deletion



# Benefit #2: Fast Operations



❖ Efficient Data Deletion

❖ User (Tuple) Removal

# Benefit #2: Fast Operations



❖ Efficient Data Deletion

❖ User (Tuple) Removal

❖ Fast Deletion

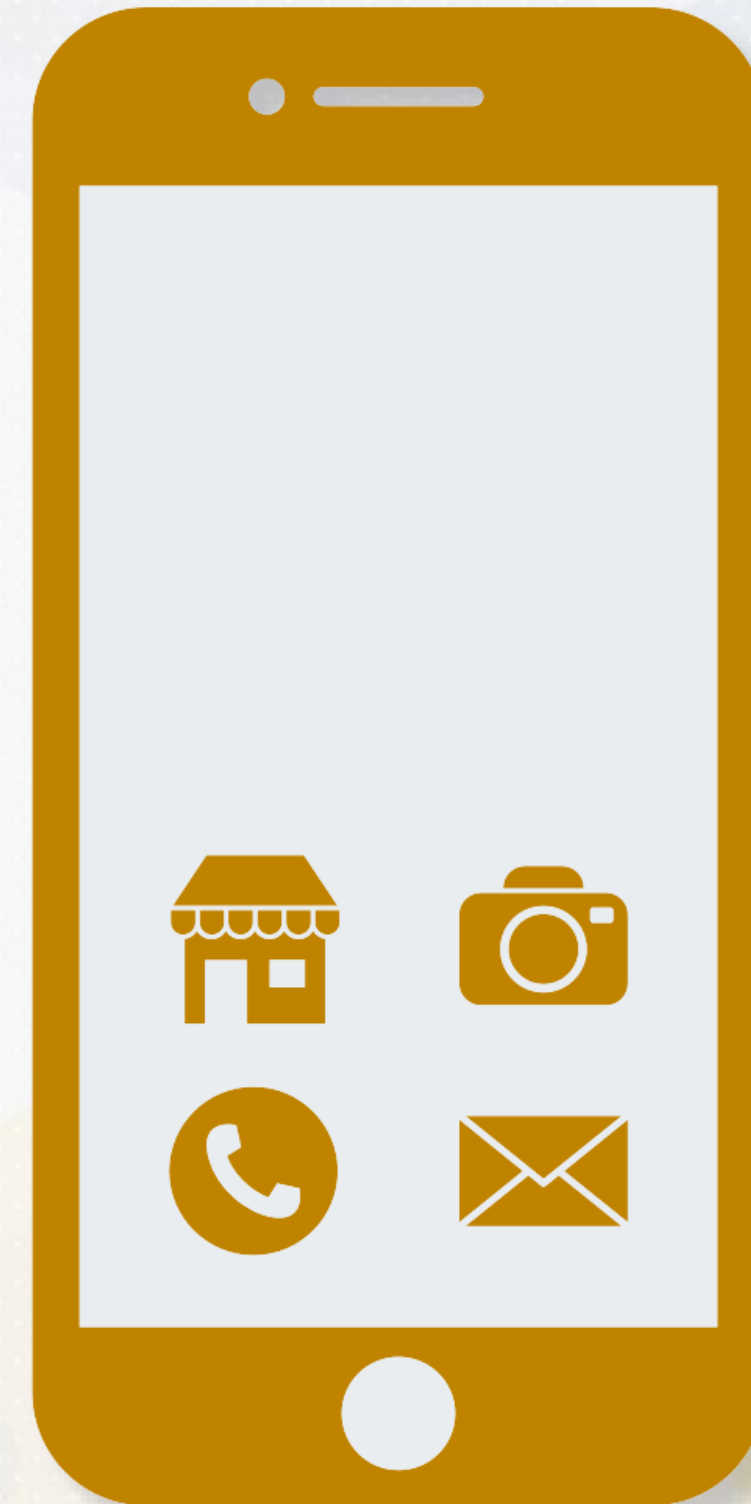
# Benefit #3: Fast Queries

# Benefit #3: Fast Queries

Index Database

# Benefit #3: Fast Queries

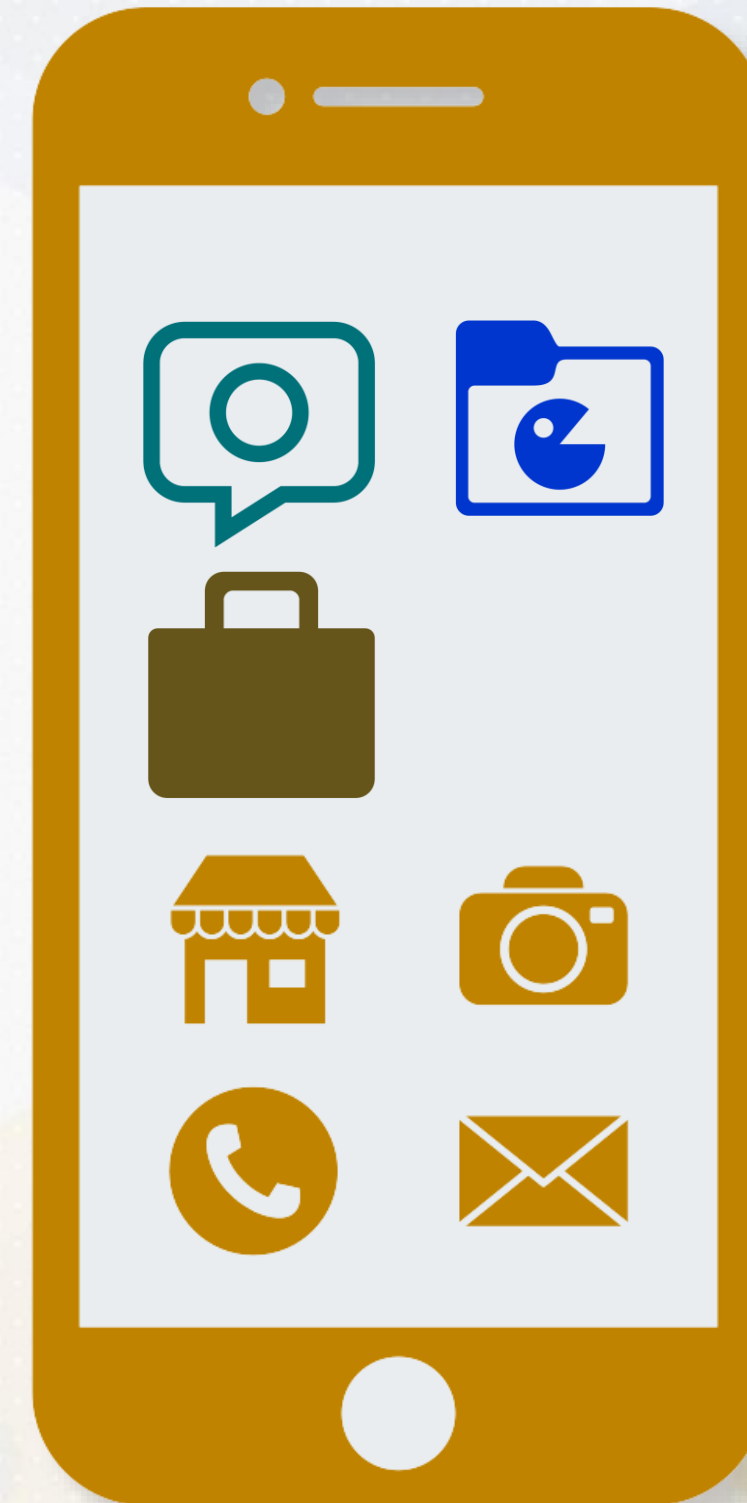
Index Database



# Benefit #3: Fast Queries

Index Database

Apps in labeled  
folders

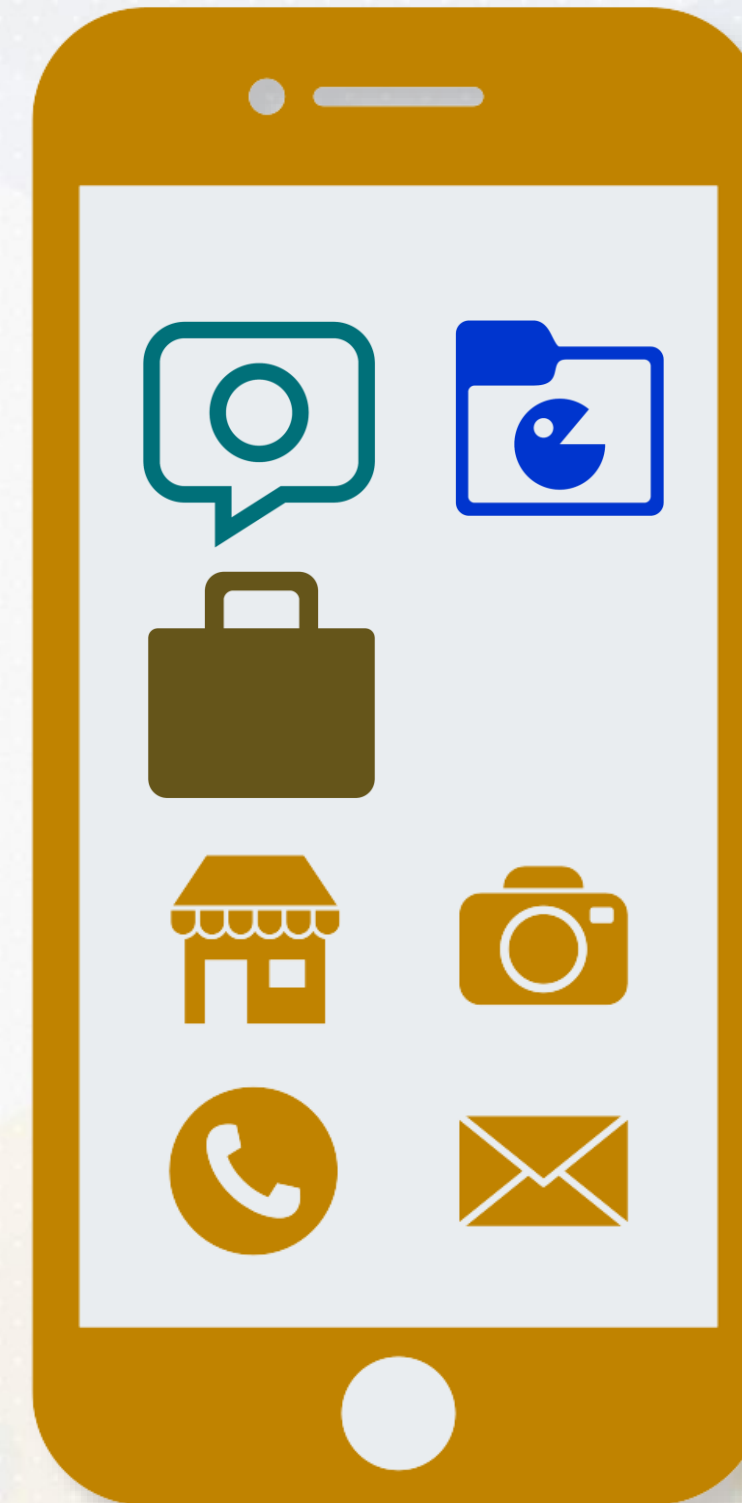


# Benefit #3: Fast Queries

Index Database

Apps in labeled  
folders

Location-based  
index



# Benefit #3: Fast Queries

| UserName,          | Location    |
|--------------------|-------------|
| Timothée Chalamet, | Paris       |
| Lana Condor,       | Los Angeles |
| Liu Yifei,         | Beijing     |
| Burna Boy,         | Lagos       |
| Kriti Sanon,       | Mumbai      |

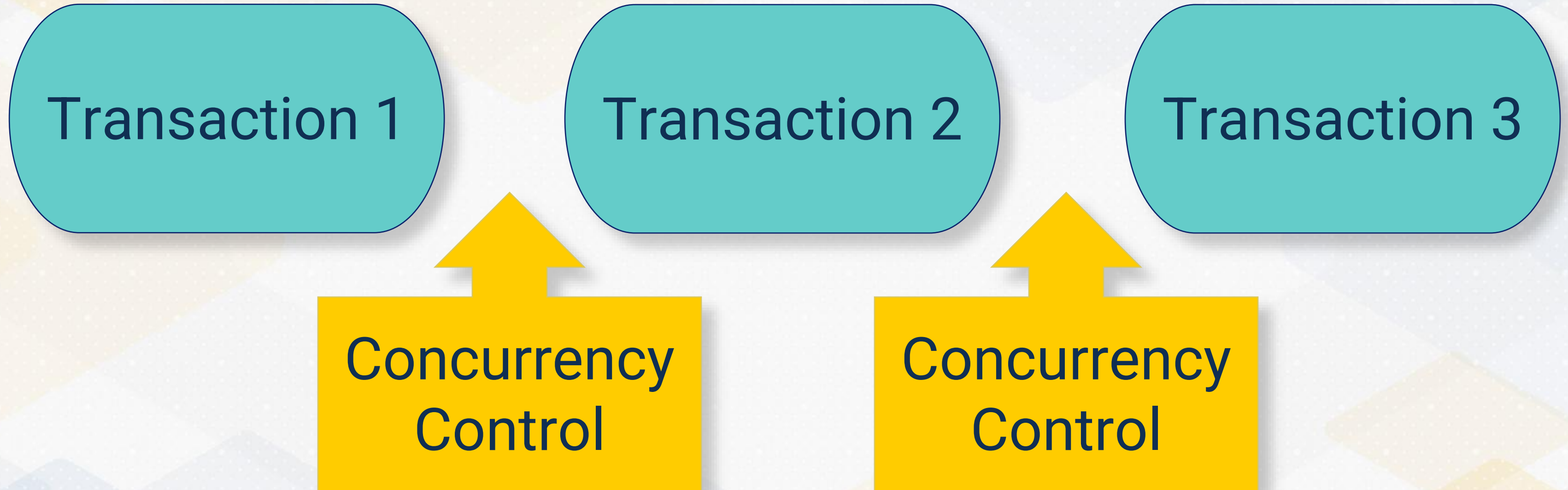
# Benefit #3: Fast Queries

```
SELECT *  
FROM Users  
WHERE LOCATION = 'Mumbai';
```

| UserName,          | Location    |
|--------------------|-------------|
| Timothée Chalamet, | Paris       |
| Lana Condor,       | Los Angeles |
| Liu Yifei,         | Beijing     |
| Burna Boy,         | Lagos       |
| Kriti Sanon,       | Mumbai      |

# Benefit #4: Concurrent Updates

# Benefit #4: Concurrent Updates



USER 1



Timothée Chalamet,  
Paris

USER 2



Timothée Chalamet,  
Paris

USER 1



Xavier Laurent,  
Paris

USER 2



Timothée Chalamet,  
Paris

USER 1



Xavier Laurent,  
Paris

USER 2



Timothée Chalamet,  
New York

USER 1



Xavier Laurent,  
Paris

USER 2



Xavier Laurent,  
New York

USER 2

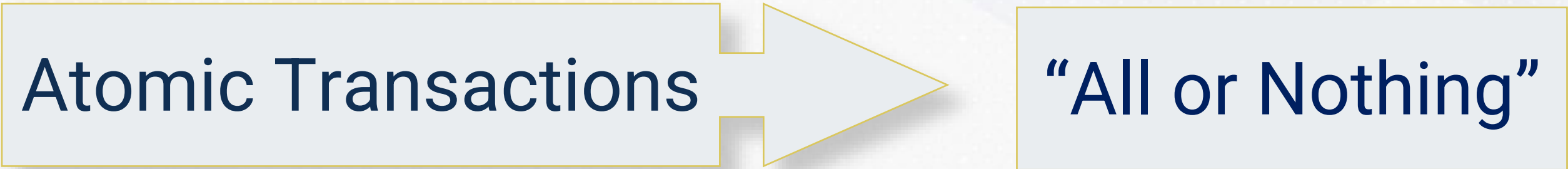


Timothée Chalamet,  
New York

# Benefit #5: Handling Failures

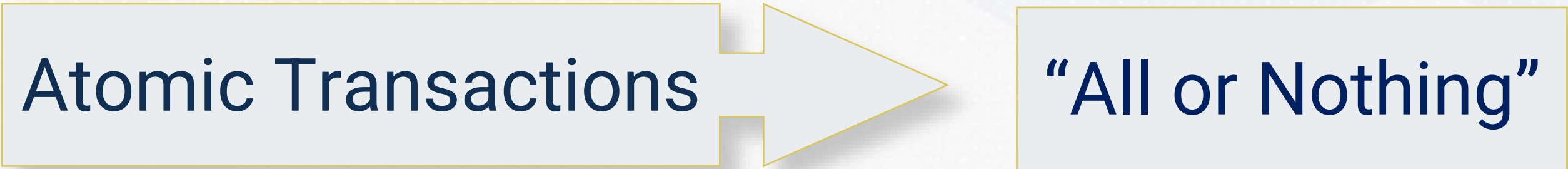
| UserName | Location | Country |
|----------|----------|---------|
|          |          |         |
|          |          |         |
|          |          |         |
|          |          |         |
|          |          |         |

# Benefit #5: Handling Failures



| UserName | Location | Country |
|----------|----------|---------|
|          |          |         |
|          |          |         |
|          |          |         |
|          |          |         |
|          |          |         |

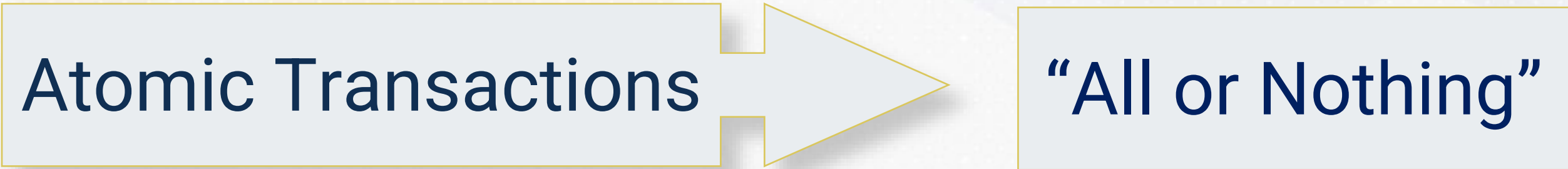
# Benefit #5: Handling Failures



| UserName          | Location    | Country |
|-------------------|-------------|---------|
| Timothée Chalamet | Paris       | France  |
| Lana Condor       | Los Angeles | USA     |
| Liu Lifei         | Beijing     | China   |
| Burna Boy         | Lagos       |         |
| Kriti Sanon       | Mumbai      |         |



# Benefit #5: Handling Failures



| UserName          | Location    | Country |
|-------------------|-------------|---------|
| Timothée Chalamet | Paris       | France  |
| Lana Condor       | Los Angeles | USA     |
| Liu Lifei         | Beijing     | China   |
| Burna Boy         | Lagos       |         |
| Kriti Sanon       | Mumbai      |         |



| UserName          | Location    |
|-------------------|-------------|
| Timothée Chalamet | Paris       |
| Lana Condor       | Los Angeles |
| Liu Yifei         | Beijing     |
| Burna Boy         | Lagos       |
| Kriti Sanon       | Mumbai      |

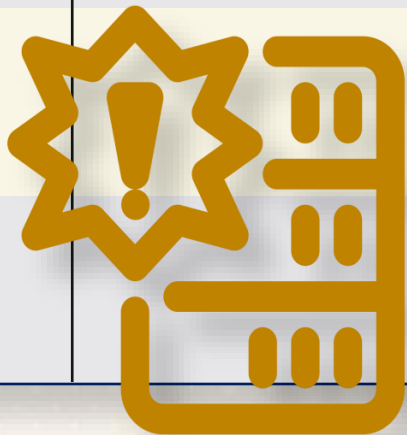
# Benefit #5: Handling Failures

| UserName          | Location    | Country |
|-------------------|-------------|---------|
| Timothée Chalamet | Paris       | France  |
| Lana Condor       | Los Angeles | USA     |
| Liu Lifei         | Beijing     | China   |
| Burna Boy         | Lagos       |         |
| Kriti Sanon       | Mumbai      |         |



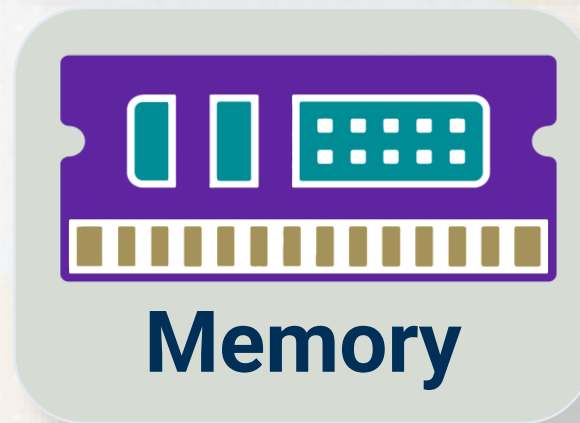
# Benefit #5: Handling Failures

| UserName          | Location    | Country |
|-------------------|-------------|---------|
| Timothée Chalamet | Paris       |         |
| Lana Condor       | Los Angeles |         |
| Liu Lifei         | Beijing     |         |
| Burna Boy         | Lagos       |         |
| Kriti Sanon       | Mumbai      |         |



# Benefit #6: Memory Management

*Faster access  
- not durable*



|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |

*Cached Pages*



*Slower access  
- but durable*

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

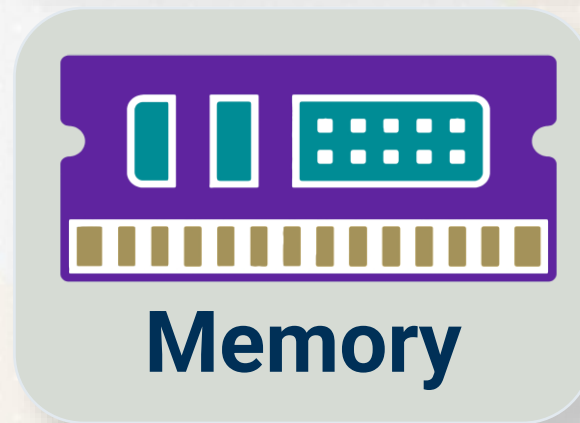
*Database*



*Transaction Log*

# Benefit #6: Memory Management

*Faster access  
- not durable*



|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |

*Cached Pages*



*Slower access  
- but durable*

|   |  |  |  |
|---|--|--|--|
|   |  |  |  |
| 1 |  |  |  |
|   |  |  |  |
|   |  |  |  |
|   |  |  |  |

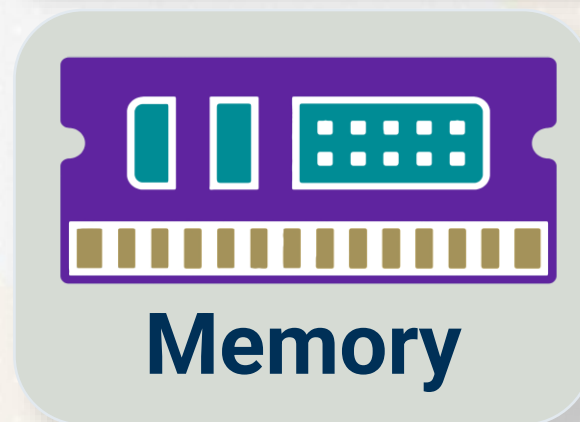
*Database*



*Transaction Log*

# Benefit #6: Memory Management

*Faster access  
- not durable*



|   |  |  |
|---|--|--|
|   |  |  |
| 2 |  |  |
|   |  |  |

*Cached Pages*



*Slower access  
- but durable*

|   |  |  |  |
|---|--|--|--|
|   |  |  |  |
| 1 |  |  |  |
|   |  |  |  |
|   |  |  |  |
|   |  |  |  |

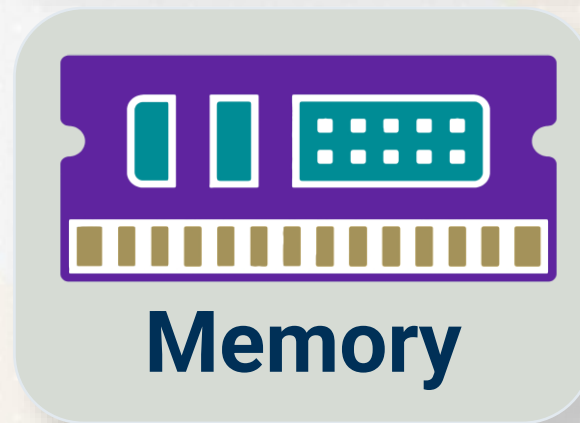
*Database*



*Transaction Log*

# Benefit #6: Memory Management

*Faster access  
- not durable*



|   |  |   |
|---|--|---|
|   |  |   |
| 2 |  | 3 |
|   |  |   |

*Cached Pages*



*Slower access  
- but durable*

|   |  |  |  |
|---|--|--|--|
|   |  |  |  |
| 1 |  |  |  |
|   |  |  |  |
|   |  |  |  |
|   |  |  |  |

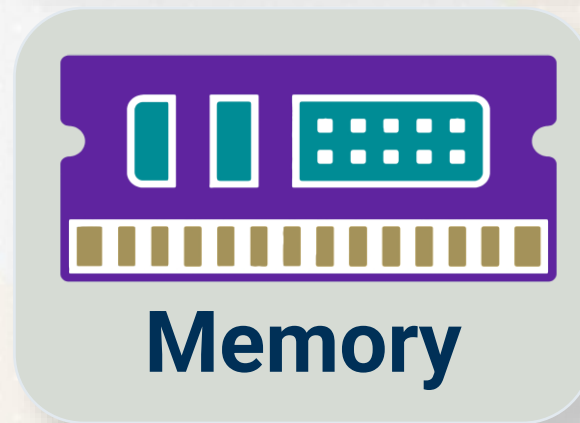
*Database*



*Transaction Log*

# Benefit #6: Memory Management

*Faster access  
- not durable*



|   |  |   |
|---|--|---|
|   |  |   |
| 2 |  | 3 |
|   |  |   |

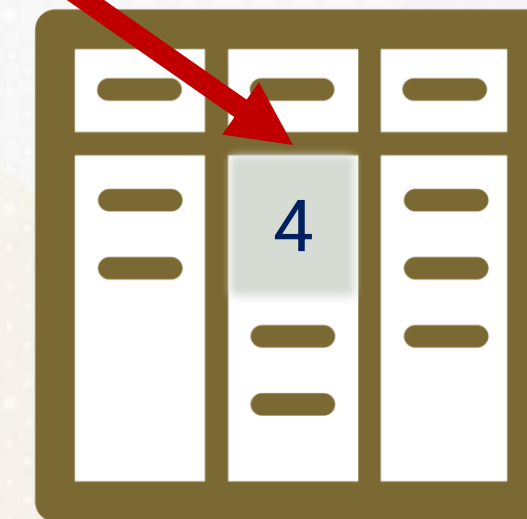
*Cached Pages*



*Slower access  
- but durable*

|   |  |  |  |
|---|--|--|--|
|   |  |  |  |
| 1 |  |  |  |
|   |  |  |  |
|   |  |  |  |
|   |  |  |  |

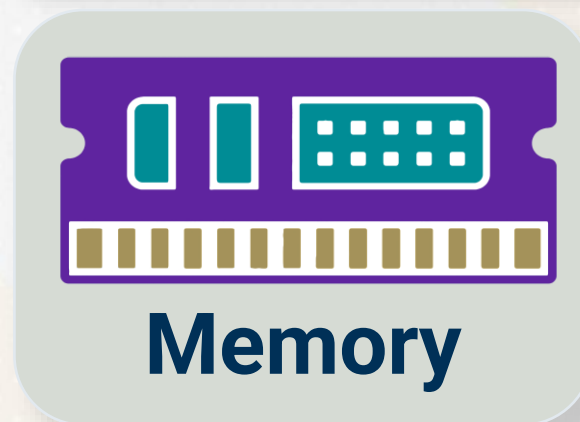
*Database*



*Transaction Log*

# Benefit #6: Memory Management

*Faster access  
- not durable*



|   |  |   |
|---|--|---|
|   |  |   |
| 2 |  | 3 |
|   |  |   |

*Cached Pages*



*Slower access  
- but durable*

|   |  |  |   |
|---|--|--|---|
|   |  |  |   |
| 1 |  |  |   |
|   |  |  | 5 |
|   |  |  |   |
|   |  |  |   |

*Database*

|  |   |  |
|--|---|--|
|  |   |  |
|  | 4 |  |
|  |   |  |
|  |   |  |
|  |   |  |

*Transaction Log*

# Benefit #7: Usability

| UserName          | Location    |
|-------------------|-------------|
| Timothée Chalamet | Paris       |
| Lana Condor       | Los Angeles |
| Liu Yifei         | Beijing     |
| Burna Boy         | Lagos       |
| Kriti Sanon       | Mumbai      |

# Benefit #7: Usability

| UserName          | Location    |
|-------------------|-------------|
| Timothée Chalamet | Paris       |
| Lana Condor       | Los Angeles |
| Liu Yifei         | Beijing     |
| Burna Boy         | Lagos       |
| Kriti Sanon       | Mumbai      |

# Benefit #7: Usability

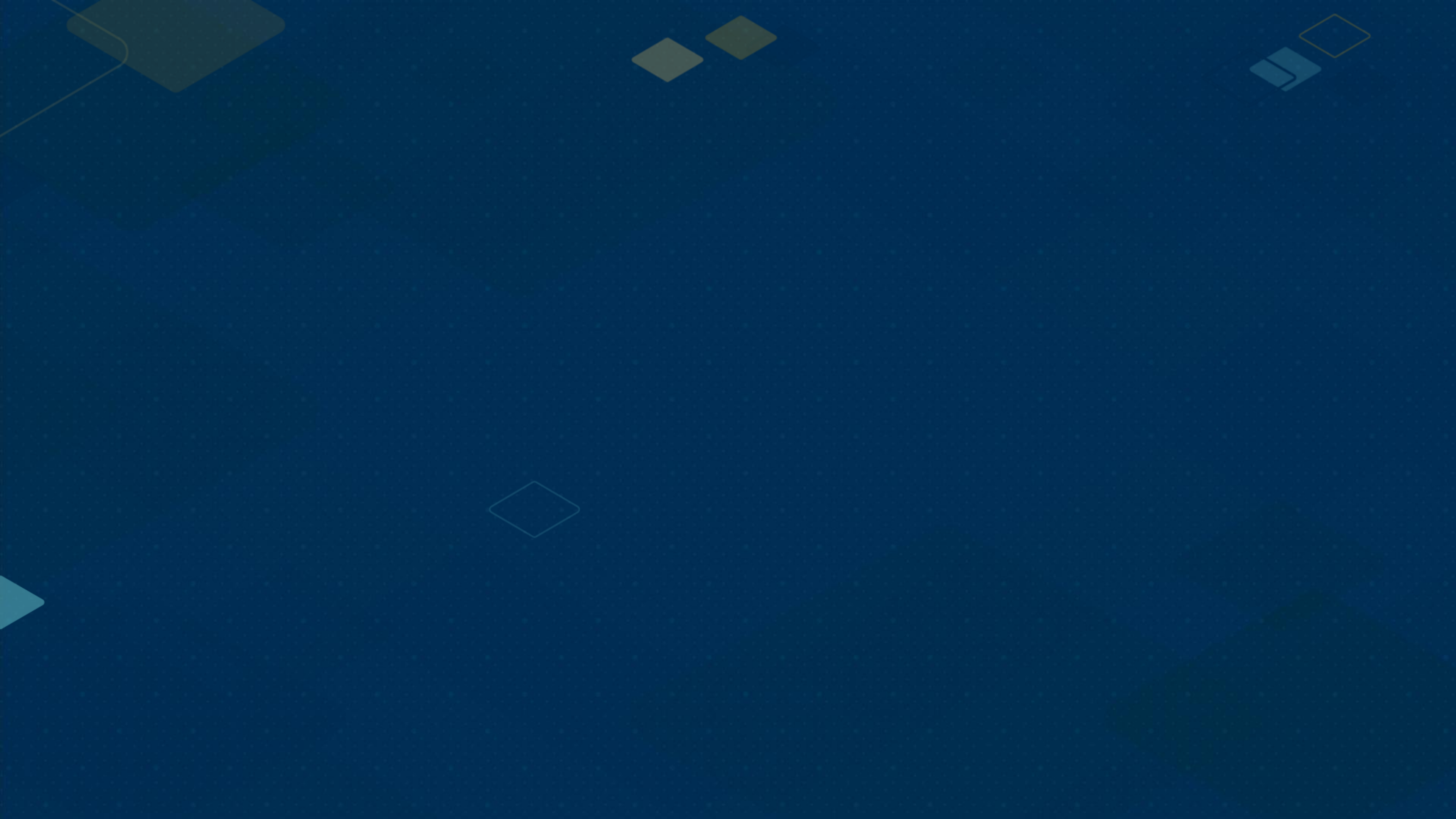
```
SELECT PostContent, CommentText  
FROM Posts  
JOIN Interactions ON Posts.PostID = Interactions.PostID  
WHERE Interactions.UserName = 'Lana Condor'  
AND Interactions.ReactionType = 'Comment';
```

# Benefit #7: Usability

```
SELECT PostContent, CommentText  
FROM Posts  
JOIN Interactions ON Posts.PostID = Interactions.PostID  
WHERE Interactions.UserName = 'Lana Condor'  
AND Interactions.ReactionType = 'Comment';
```

SQL = Declarative

Python, C++ = Imperative





# Relational Database





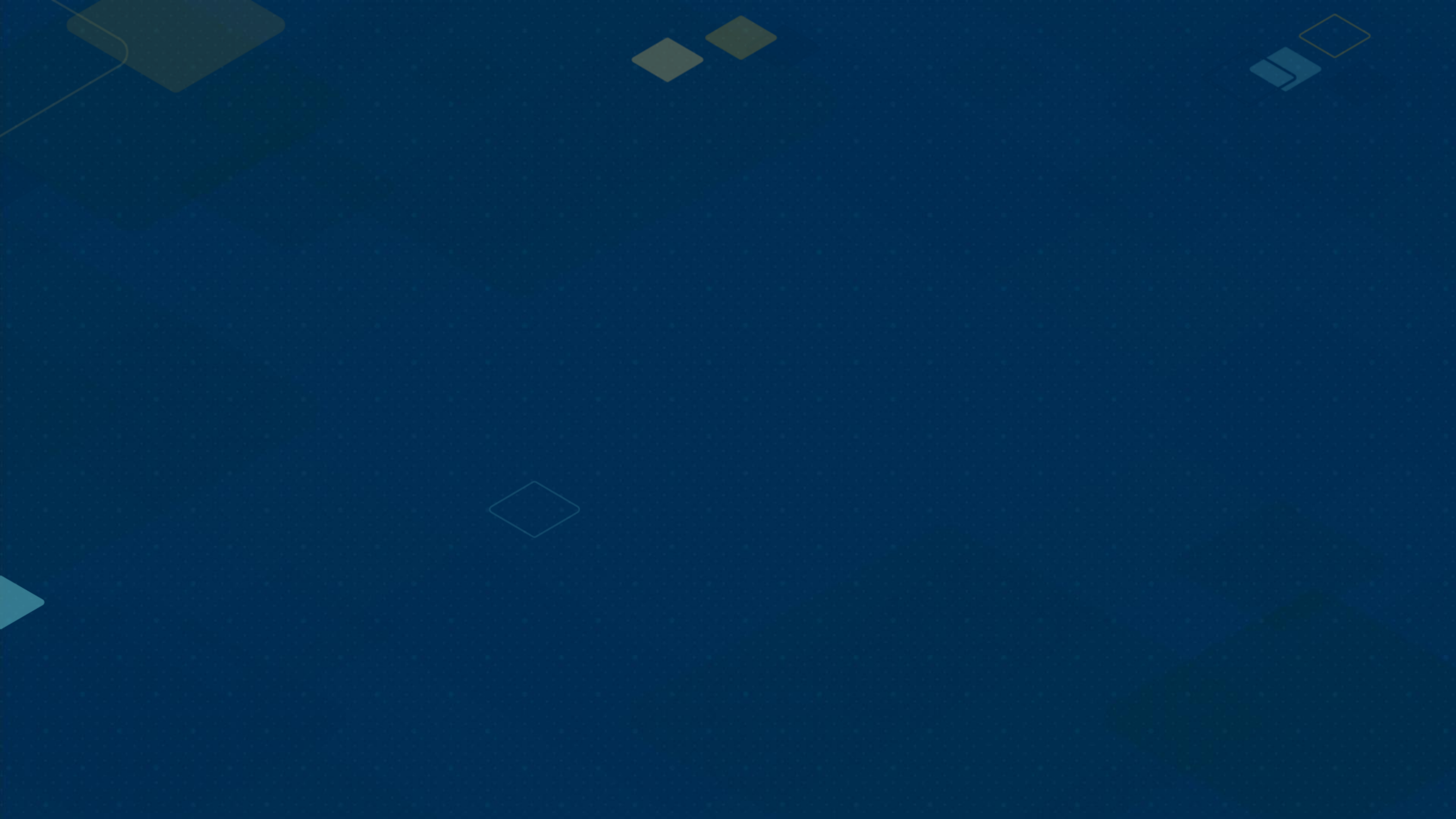
Relational  
Database

Relational  
Database  
Benefits



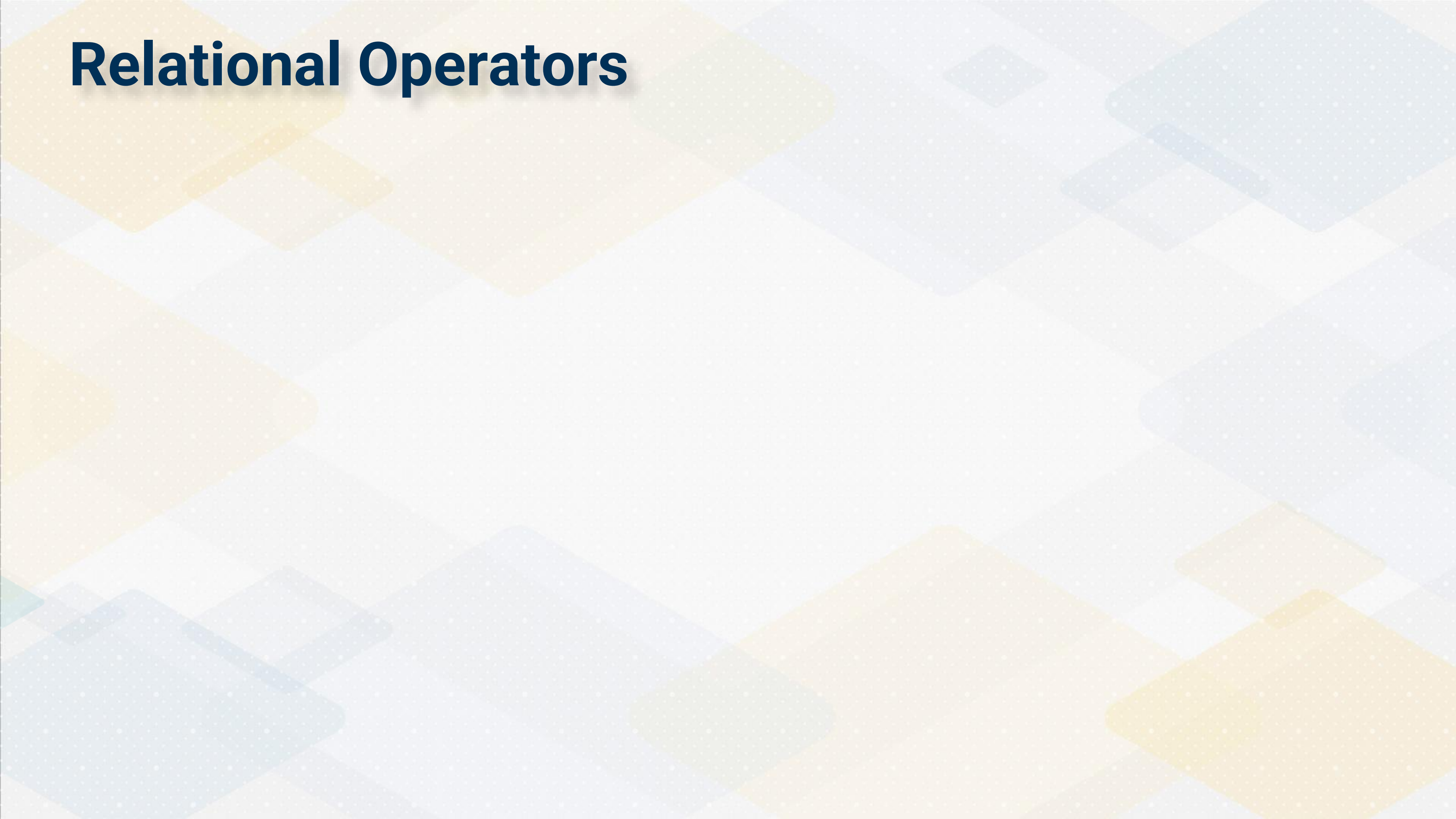
# Relational Operators





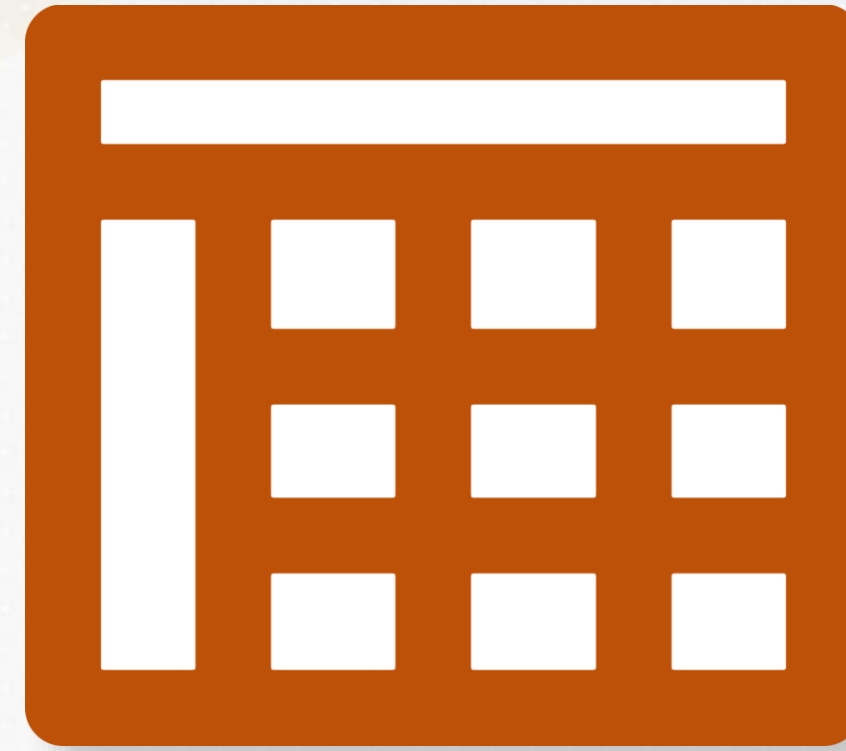
# Relational Operators

# Relational Operators

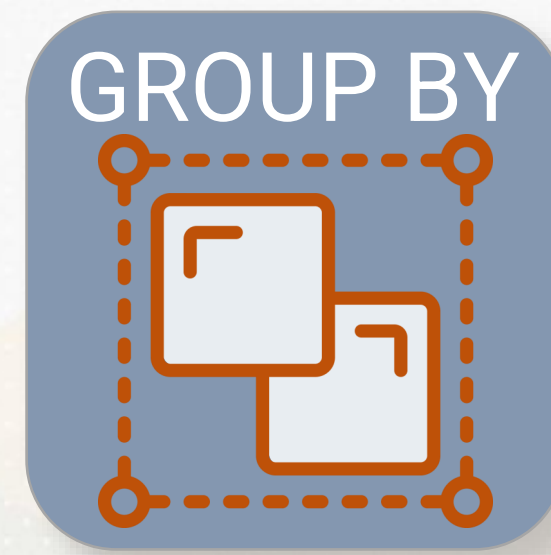
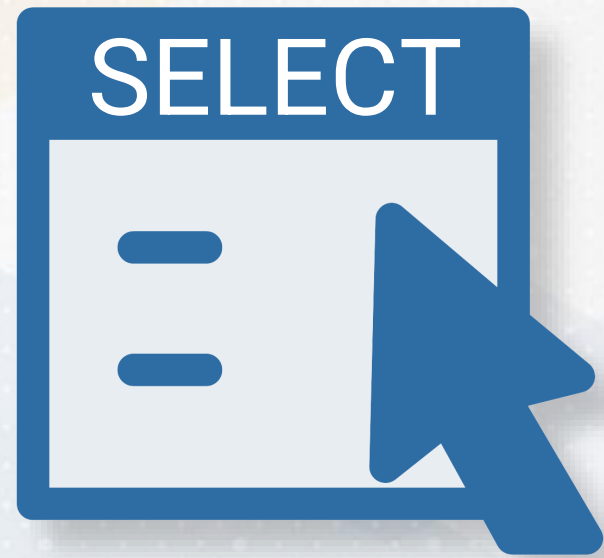
The background of the slide is an abstract geometric pattern. It consists of numerous overlapping squares and rectangles in three primary colors: yellow, blue, and green. These shapes are arranged in a way that creates a sense of depth and movement, with some shapes appearing to be in front of others. The colors are muted and the overall effect is clean and modern.

# Relational Operators

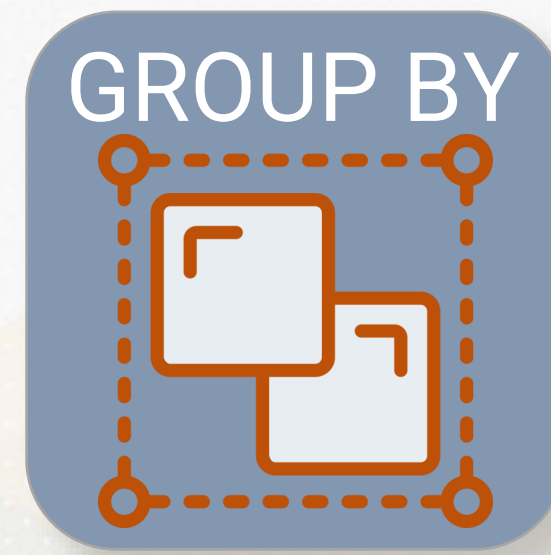
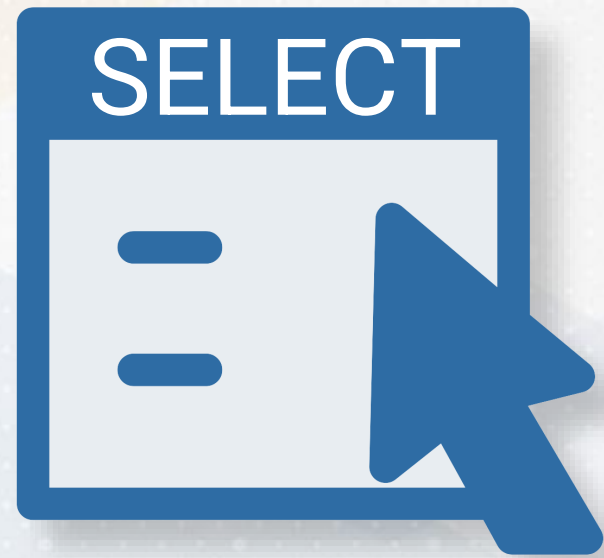
Relations



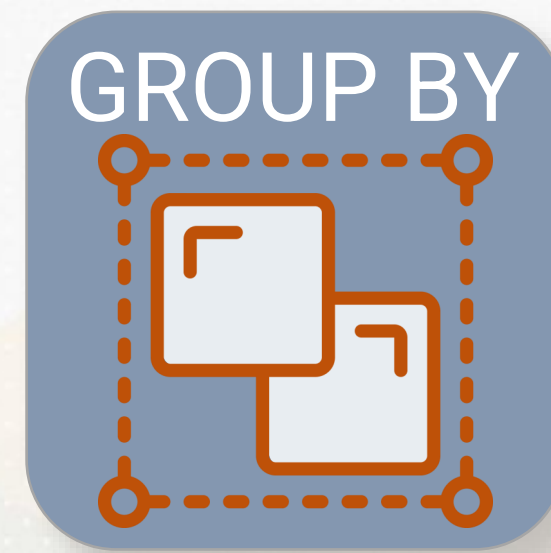
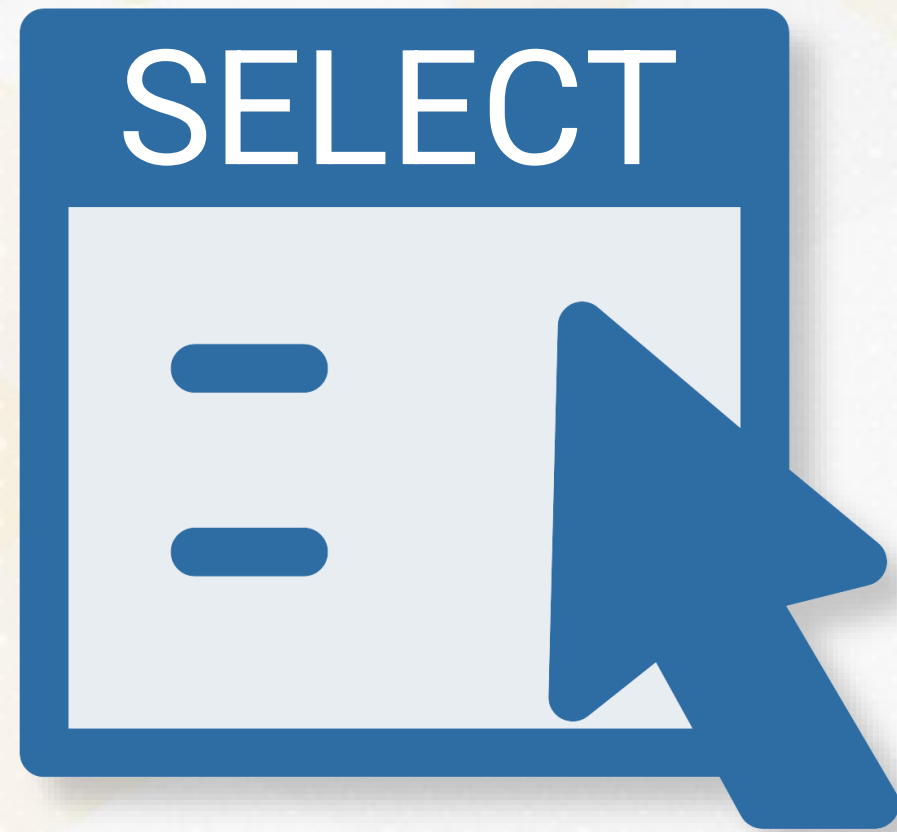
# Relational Operators



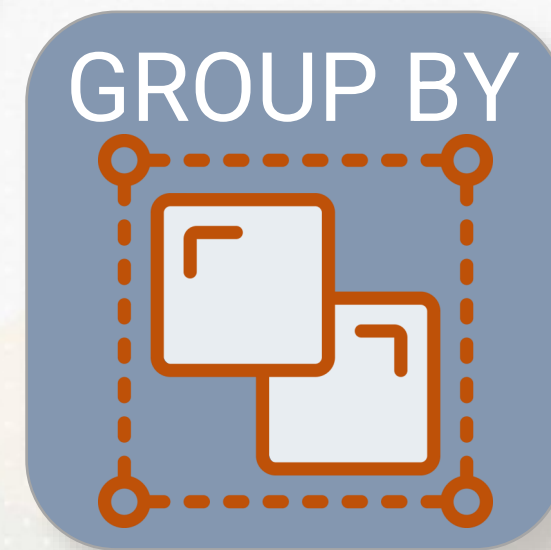
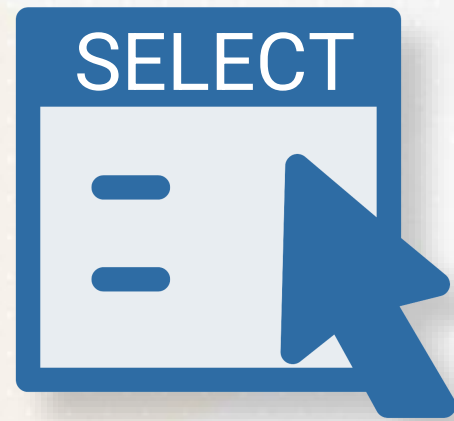
# Relational Operators



# Relational Operators



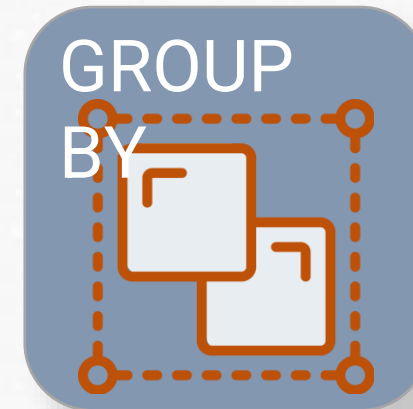
# Relational Operators



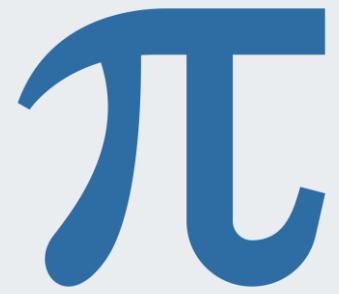
# Relational Operators



# Relational Operators



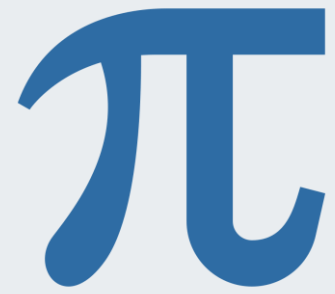
# SELECT (Projection Operator)



Select specific columns from a table

Example: Retrieve locations of all users.

# SELECT (Projection Operator)



Select specific columns from a table

Example: Retrieve locations of all users.

```
SELECT Location  
FROM Users;
```

# WHERE (Selection Operator)



Filters rows based on specified conditions

Example: Find all interactions that are "Like" reactions.

# WHERE (Selection Operator)

**σ** Filters rows based on specified conditions

Example: Find all interactions that are "Like" reactions.

```
SELECT *  
FROM Interactions  
WHERE ReactionType = 'Like';
```

# GROUP BY (Grouping Operator)

Y

- Groups rows of same values
- Used with aggregate functions like SUM

Example: Count the number of reactions that each post received.

# GROUP BY (Grouping Operator)

Y

- Groups rows of same values
- Used with aggregate functions like SUM

Example: Count the number of reactions that each post received.

```
SELECT PostID,  
       COUNT(*) AS ReactionCount  
FROM Interactions  
GROUP BY PostID;
```

# SUM (Aggregation Operator)



- Adds group values
- Defined by **GROUP BY** clause

Example: Total number of posts made by each user, grouping the results by UserID.

# SUM (Aggregation Operator)

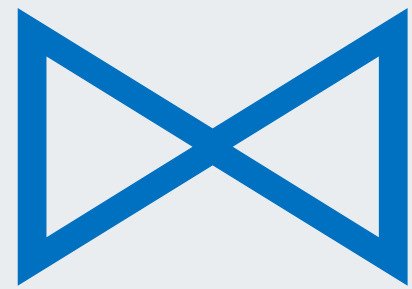


- Adds group values
- Defined by **GROUP BY** clause

Example: Total number of posts made by each user, grouping the results by UserID.

```
SELECT UserID,  
       COUNT(PostID) AS TotalPosts  
FROM Posts  
GROUP BY UserID;
```

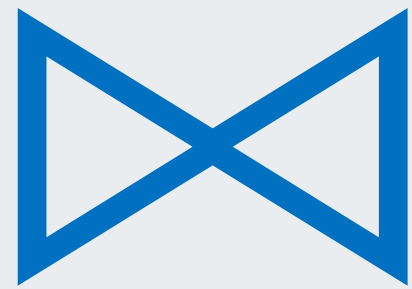
# JOIN (Join Operator)



- Links rows from two different tables
- Combine information from both

Example: Engagement received by each post along with its content.

# JOIN (Join Operator)



- Links rows from two different tables
- Combine information from both

Example: Engagement received by each post along with its content.

```
SELECT Posts.PostContent,  
       COUNT(Interactions.ReactionType) AS TotalInteractions  
FROM Posts  
JOIN Interactions ON Posts.PostID = Interactions.PostID  
GROUP BY Posts.PostID;
```



# Relational Algebra



# Relational Algebra



# Relational Algebra

Theoretical  
Foundation

Combine  
Operators

Creating  
a Query

# Relational Algebra



# Power of SQL

The background of the slide is an abstract pattern of overlapping squares in various shades of yellow, light blue, and pale green. The squares are arranged in a way that creates a sense of depth and movement, with some squares appearing more prominent than others. The overall effect is a clean, modern, and visually appealing design.

# Power of SQL

## Combine Operators

```
SELECT Interactions.PostID, COUNT(*) AS TotalLikes,  
       GROUP_CONCAT(DISTINCT Users.UserID  
                     || '-' || Users.Username)  
       AS UsersWhoLiked  
FROM Interactions  
JOIN Users ON Interactions.UserID = Users.UserID  
WHERE Interactions.ReactionType = 'Like'  
GROUP BY Interactions.PostID  
ORDER BY TotalLikes DESC  
LIMIT 1;
```

# Power of SQL

Combine Operators

Filter Interactions

```
SELECT Interactions.PostID, COUNT(*) AS TotalLikes,  
       GROUP_CONCAT(DISTINCT Users.UserID  
                     || '-' || Users.Username)  
       AS UsersWhoLiked  
FROM Interactions  
JOIN Users ON Interactions.UserID = Users.UserID  
WHERE Interactions.ReactionType = 'Like'  
GROUP BY Interactions.PostID  
ORDER BY TotalLikes DESC  
LIMIT 1;
```

# Power of SQL

Combine Operators

Filter Interactions

Combine Tables

```
SELECT Interactions.PostID, COUNT(*) AS TotalLikes,  
       GROUP_CONCAT(DISTINCT Users.UserID  
                     || '-' || Users.Username)  
       AS UsersWhoLiked  
FROM Interactions  
JOIN Users ON Interactions.UserID = Users.UserID  
WHERE Interactions.ReactionType = 'Like'  
GROUP BY Interactions.PostID  
ORDER BY TotalLikes DESC  
LIMIT 1;
```

# Power of SQL

Combine Operators

Filter Interactions

Combine Tables

Group & Count Results

```
SELECT Interactions.PostID, COUNT(*) AS TotalLikes,  
       GROUP_CONCAT(DISTINCT Users.UserID  
                     || '-' || Users.Username)  
       AS UsersWhoLiked  
FROM Interactions  
JOIN Users ON Interactions.UserID = Users.UserID  
WHERE Interactions.ReactionType = 'Like'  
GROUP BY Interactions.PostID  
ORDER BY TotalLikes DESC  
LIMIT 1;
```

# Power of SQL

Combine Operators

Filter Interactions

Combine Tables

Group & Count Results

Project Fields

```
SELECT Interactions.PostID, COUNT(*) AS TotalLikes,  
       GROUP_CONCAT(DISTINCT Users.UserID  
                     || '-' || Users.Username)  
       AS UsersWhoLiked  
FROM Interactions  
JOIN Users ON Interactions.UserID = Users.UserID  
WHERE Interactions.ReactionType = 'Like'  
GROUP BY Interactions.PostID  
ORDER BY TotalLikes DESC  
LIMIT 1;
```

# Power of SQL

Combine Operators

Filter Interactions

Combine Tables

Group & Count Results

Project Fields

Sort Posts

```
SELECT Interactions.PostID, COUNT(*) AS TotalLikes,  
       GROUP_CONCAT(DISTINCT Users.UserID  
                     || '-' || Users.Username)  
       AS UsersWhoLiked  
FROM Interactions  
JOIN Users ON Interactions.UserID = Users.UserID  
WHERE Interactions.ReactionType = 'Like'  
GROUP BY Interactions.PostID  
ORDER BY TotalLikes DESC  
LIMIT 1;
```

# Power of SQL

Combine Operators

Filter Interactions

Combine Tables

Group & Count Results

Project Fields

Sort Posts

```
SELECT Interactions.PostID, COUNT(*) AS TotalLikes,  
       GROUP_CONCAT(DISTINCT Users.UserID  
                     || ' - ' || Users.Username)  
       AS UsersWhoLiked  
FROM Interactions  
JOIN Users ON Interactions.UserID = Users.UserID  
WHERE Interactions.ReactionType = 'Like'  
GROUP BY Interactions.PostID  
ORDER BY TotalLikes DESC  
LIMIT 1;
```

## Results

Post 1002 liked by 3 users  
'3 - Liu Yifei,4 - Burna Boy,5 - Kriti Sanon'

# Relational Algebra

```
T TotalLikes DESC
( $\pi$  PostID, TotalLikes, UsersWhoLiked
( $\gamma$  COUNT(*)  $\rightarrow$  TotalLikes,
  GROUP_CONCAT(..)  $\rightarrow$  UsersWhoLiked
( $\sigma$  ReactionType='Like'
  (Interactions)  $\bowtie$  Users)
)
)
```

# Relational Algebra

## Filters Interactions

```
T TotalLikes DESC
( $\pi$  PostID, TotalLikes, UsersWhoLiked
 ( $\gamma$  COUNT(*)  $\rightarrow$  TotalLikes,
   GROUP_CONCAT(..)  $\rightarrow$  UsersWhoLiked
 ( $\sigma$  ReactionType='Like'
  (Interactions)  $\bowtie$  Users)
 )
 )
```

# Relational Algebra

Filters Interactions

Combine Tables

```
T TotalLikes DESC
( $\pi$  PostID, TotalLikes, UsersWhoLiked
 ( $\gamma$  COUNT(*)  $\rightarrow$  TotalLikes,
   GROUP_CONCAT(..)  $\rightarrow$  UsersWhoLiked
 ( $\sigma$  ReactionType='Like'
  (Interactions)  $\bowtie$  Users)
 )
 )
```

# Relational Algebra

Filters Interactions

Combine Tables

Group & Aggregate

```
T TotalLikes DESC
( $\pi$  PostID, TotalLikes, UsersWhoLiked
 ( $\gamma$  COUNT(*)  $\rightarrow$  TotalLikes,
   GROUP_CONCAT(..)  $\rightarrow$  UsersWhoLiked
 ( $\sigma$  ReactionType='Like'
  (Interactions)  $\bowtie$  Users)
 )
 )
```

# Relational Algebra

Filters Interactions

Combine Tables

Group & Aggregate

Project Output

```
T TotalLikes DESC
( $\pi$  PostID, TotalLikes, UsersWhoLiked
 ( $\gamma$  COUNT(*) $\rightarrow$ TotalLikes,
   GROUP_CONCAT(..)  $\rightarrow$  UsersWhoLiked
 ( $\sigma$  ReactionType='Like'
  (Interactions)  $\bowtie$  Users)
 )
 )
```

# Relational Algebra

Filters Interactions

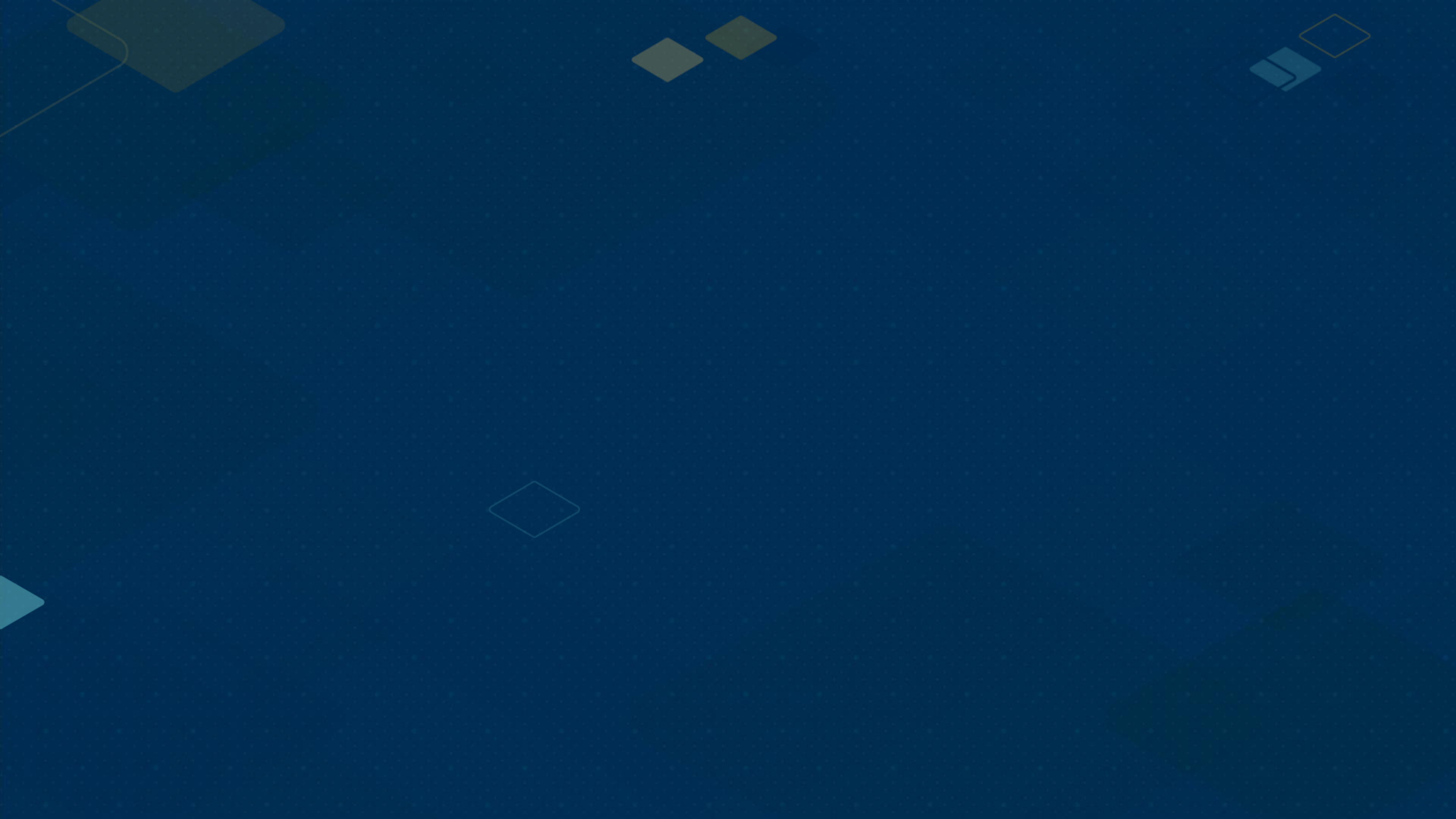
Combine Tables

Group & Aggregate

Project Output

Sort Posts

```
T TotalLikes DESC
( $\pi$  PostID, TotalLikes, UsersWhoLiked
 ( $\gamma$  COUNT(*) $\rightarrow$ TotalLikes,
   GROUP_CONCAT(..)  $\rightarrow$  UsersWhoLiked
 ( $\sigma$  ReactionType='Like'
  (Interactions)  $\bowtie$  Users)
 )
 )
```



# **Relational Operators**

**Relational  
Operators**

**Relational  
Algebra**